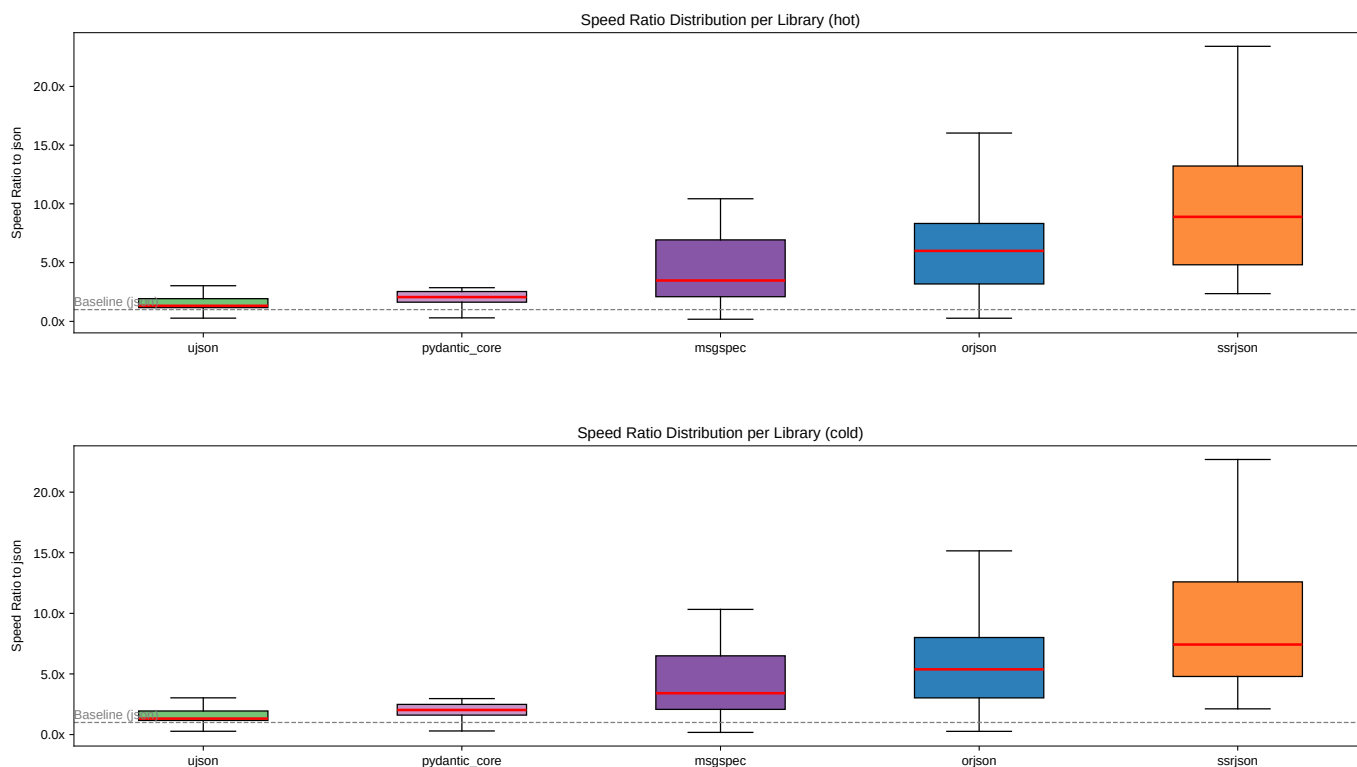


ssrJSON benchmark Report

REV: `0.0.23`
Python: `3.14.7 (main, Aug 5 2026, 10:29:49) [GCC 16.2.0]`
Orjson: `3.11.9`
MsgSpec: `0.21.1`
Ujson: `5.13.0`
PydanticCore: `2.46.4`
Generated time: 2026-10-01 18:26:51 CST
OS: Linux x86_64 7.2.7-zen1 #1-NixOS ZEN SMP PREEMPT_DYNAMIC Tue Jan 1 00:00:00 UTC 1980
SIMD flag: {'multi_lib': True, 'simd': 'AVX2'}
Chipset: 13th Gen Intel(R) Core(TM) i7-13700K
Memory: 31.107GiB
Process(GB): 0.100GiB
Locality: hot, cold (cold ring = 2x LLC)
LLC: 30.0MiB (linux-sysfs)
MinIters: 200
Statistic: median of 5 rounds; timer overhead 29ns/call
Pinned CPU: core 8 [fastest] 5400MHz siblings=8-9
Environment: governor=powersave turbo=on smt=on load=1.70 1.48 1.44
hashseed=randomized

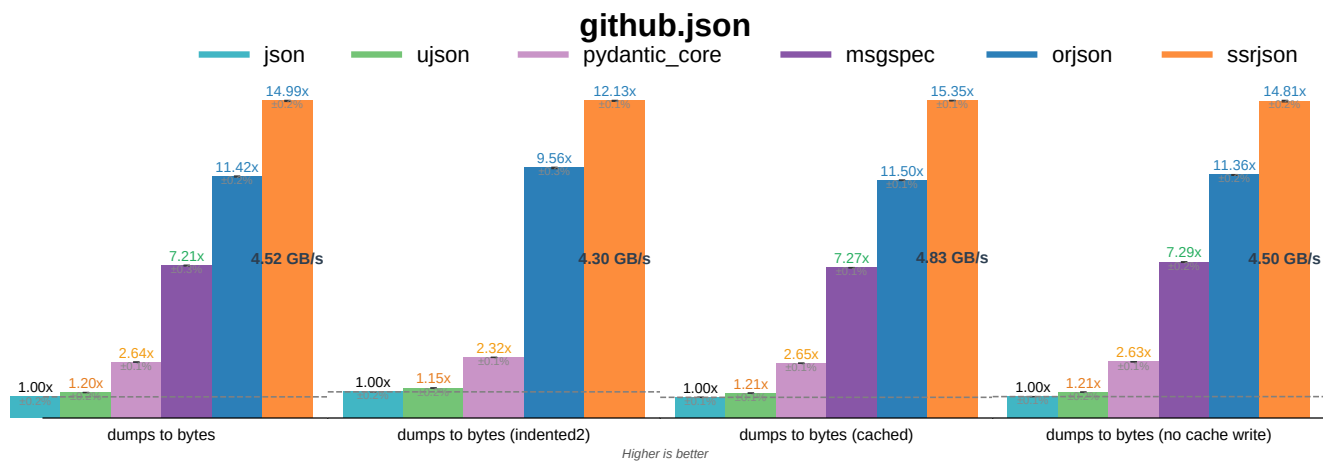
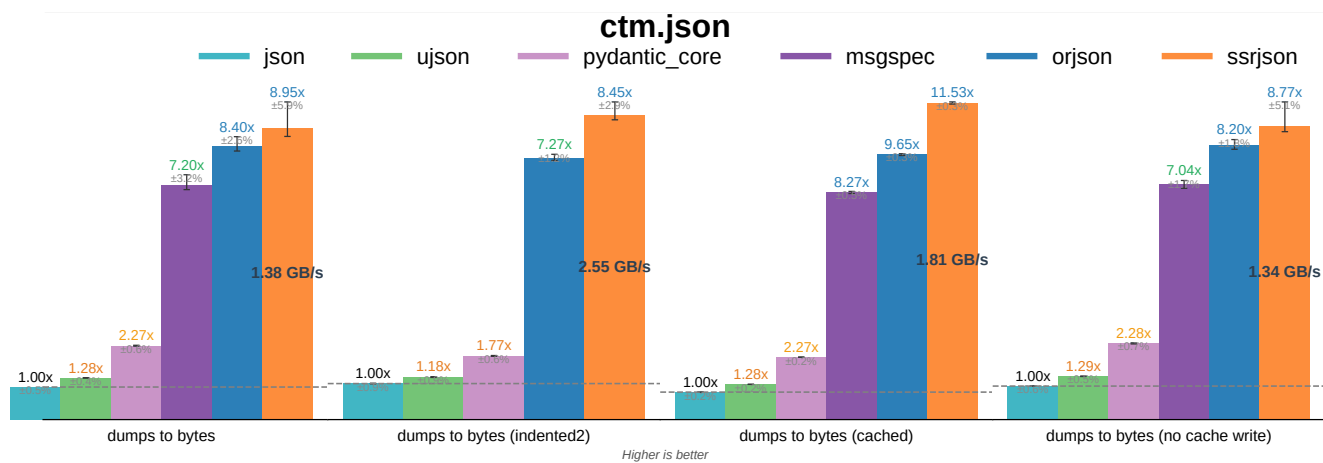
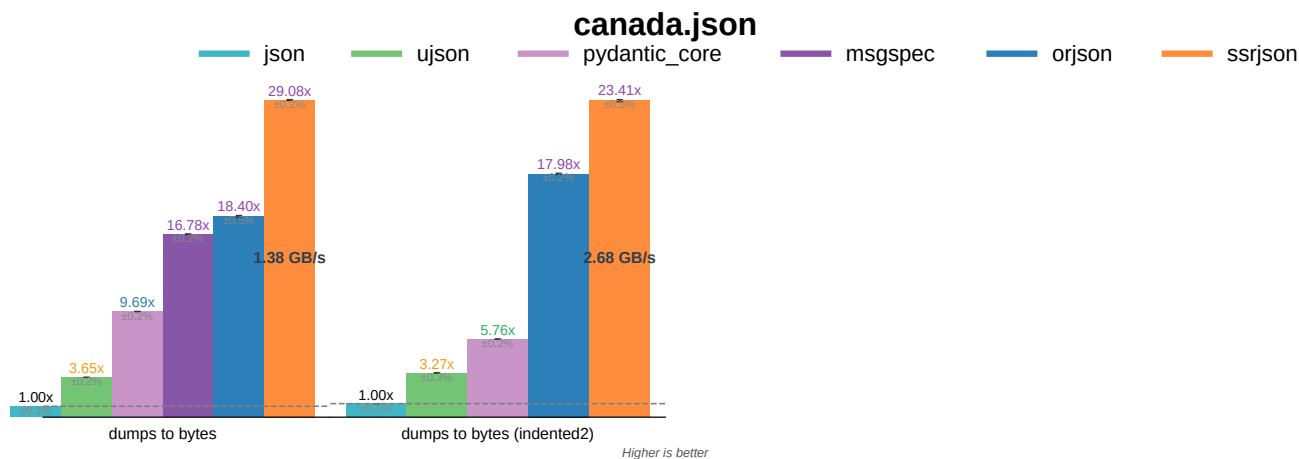
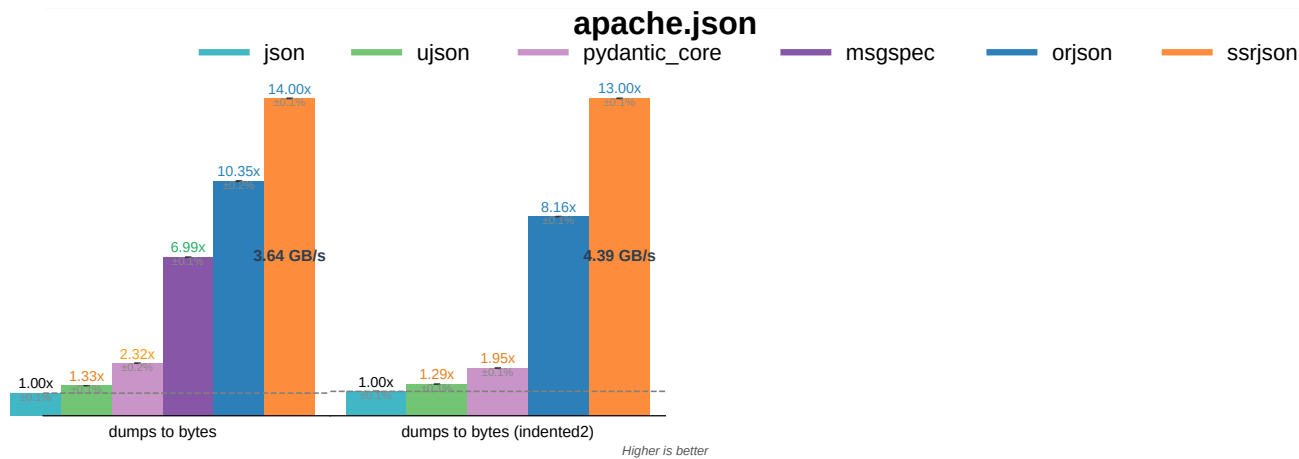
TL;DR



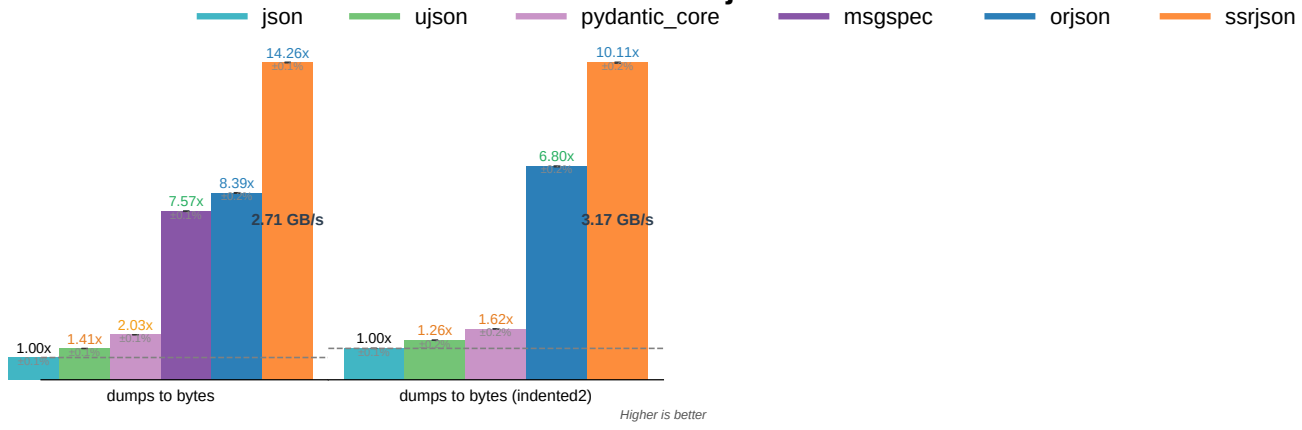
Every library in this summary runs in the configuration its wheel ships with, including ssrJSON's UTF-8 cache writing (on by default, and not switchable at all in orjson, msgspec or pydantic_core). 'dumps to bytes' is therefore the serialize-once cost and 'dumps to bytes (cached)' the steady-state repeat-dump cost; real workloads sit between them depending on how often the caller re-serializes the same object. Excluded from this summary (still charted below): 'dumps to bytes (no cache write)' -- ssrJSON is forced out of its default there to isolate what the cache write costs, which is an engine diagnostic rather than a package comparison.

UTF-8 cache break-even for ssrJSON -- how many times one object must be serialized before writing the cache pays for itself: simple_object_zh.json N=1.2; twitter.json N=1.5. Below that, ssrjson.write_utf8_cache(False) is faster; above it, the default wins. On ctm.json, github.json the cache write costs less than the measurement can resolve, so there is no crossover to report and the default needs no thought.

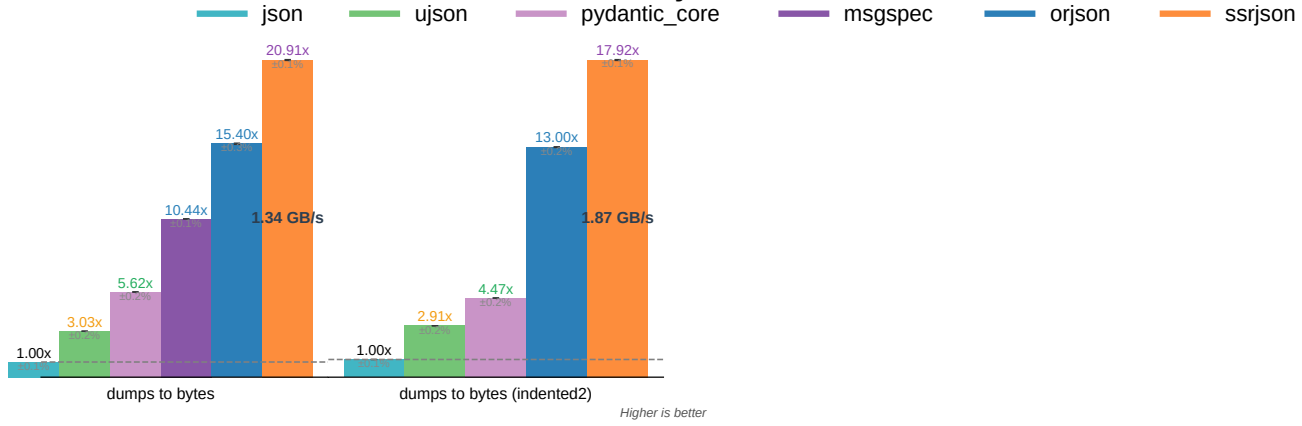
Dumps to bytes (hot)



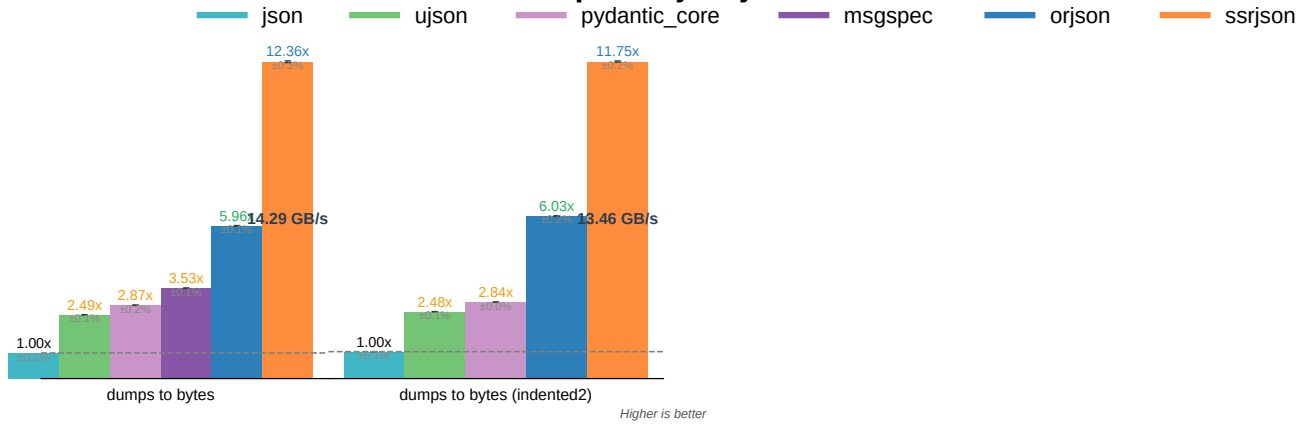
instruments.json



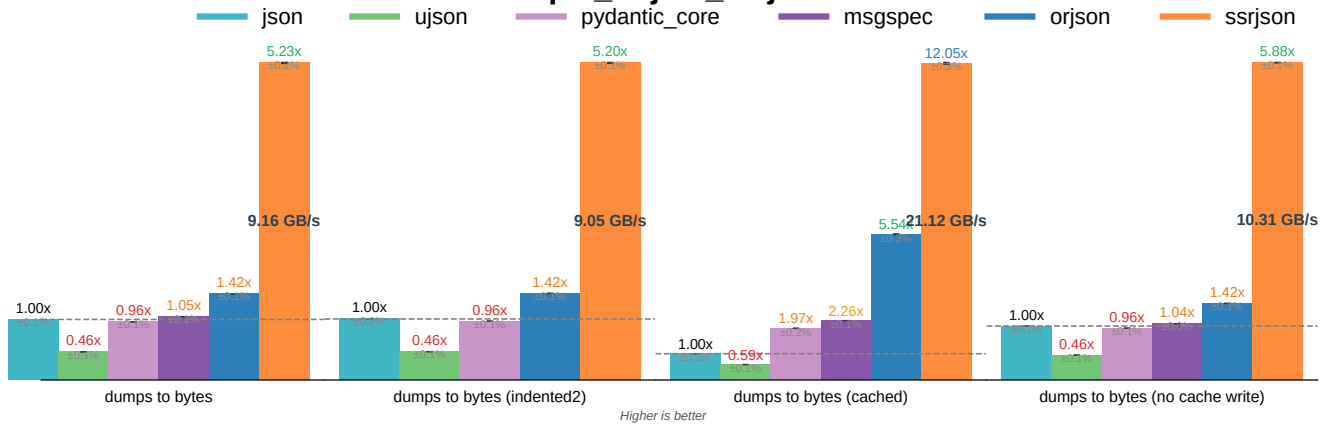
mesh.json



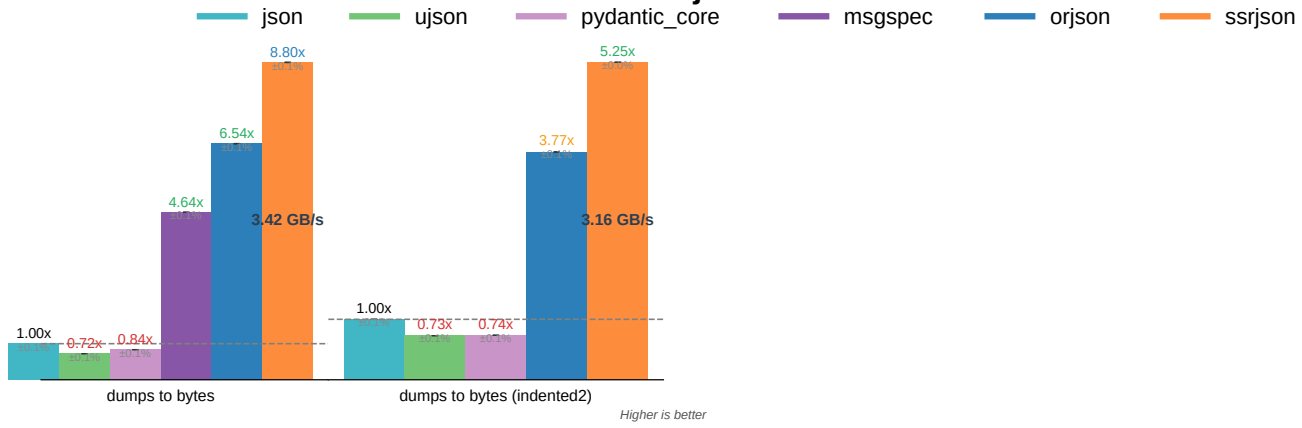
simple_object.json



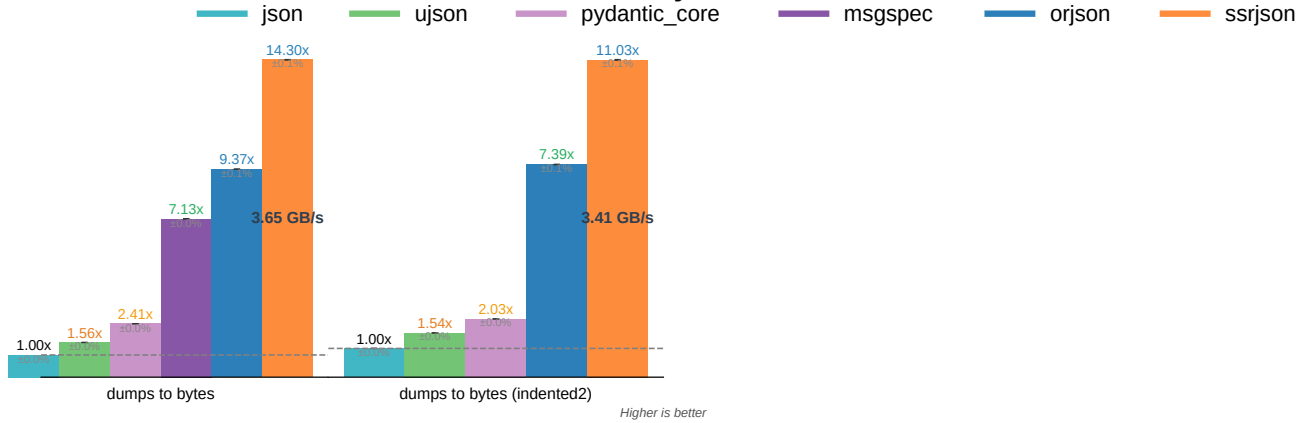
simple_object_zh.json



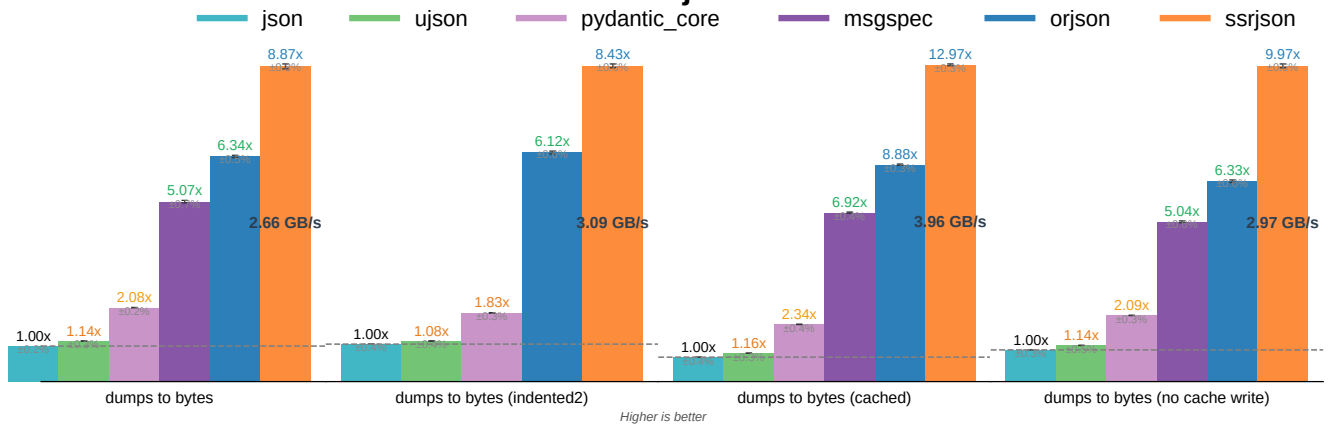
truenull.json



tweet.json

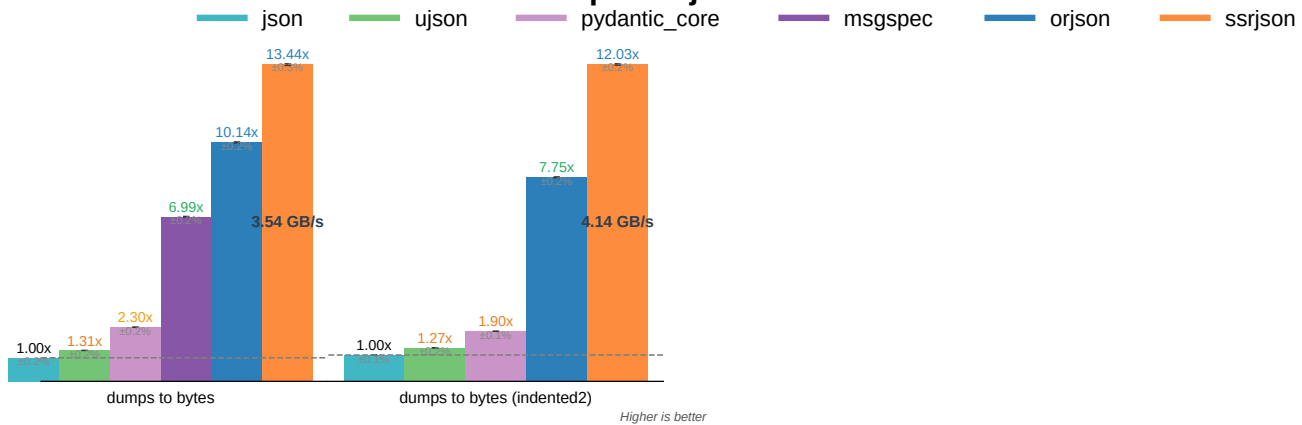


twitter.json

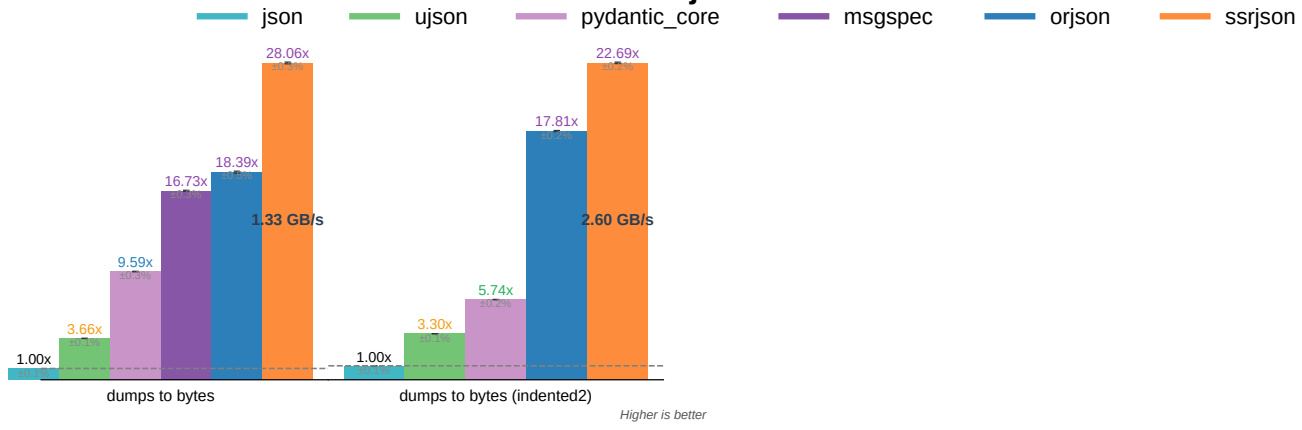


Dumps to bytes (cold)

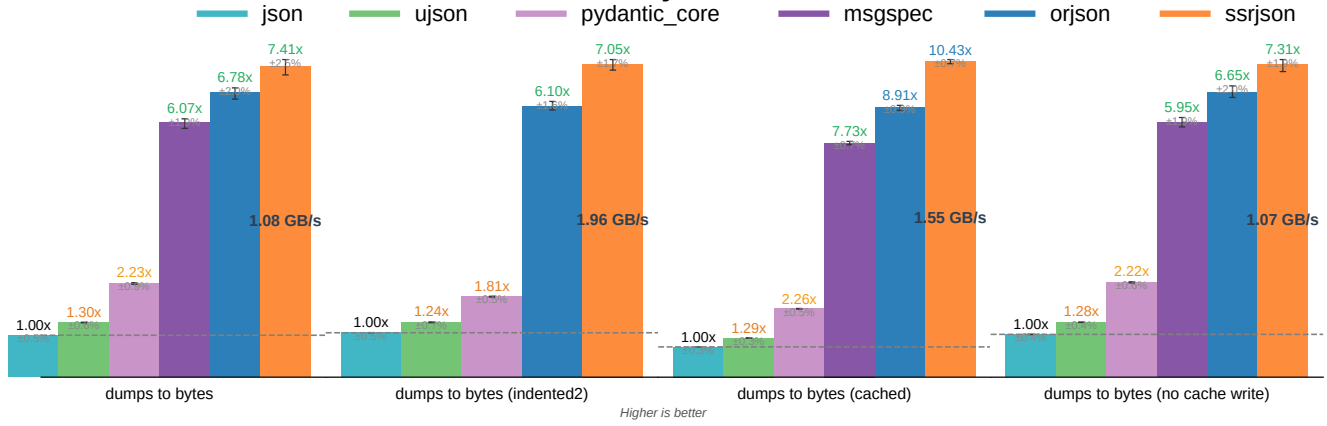
apache.json



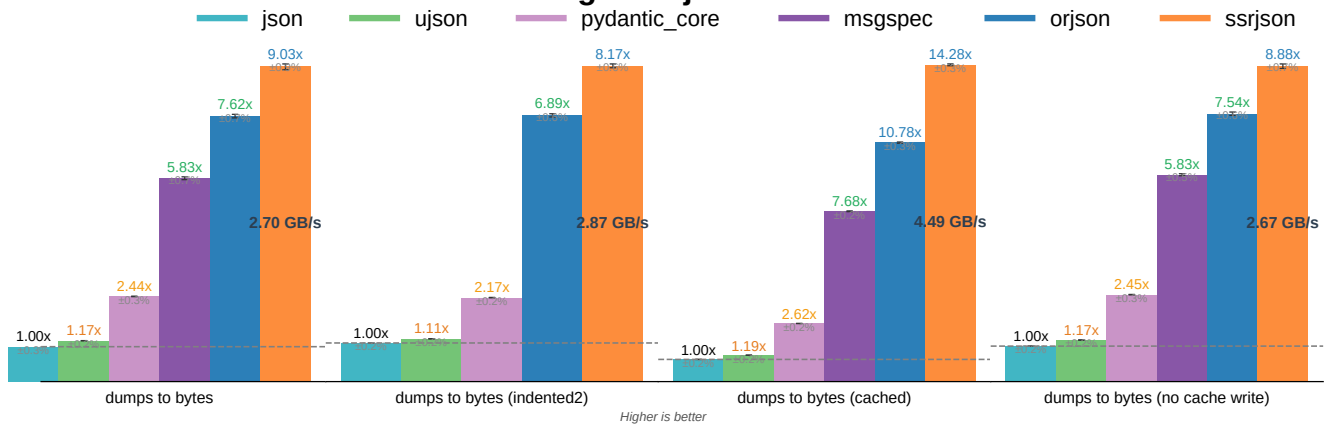
canada.json



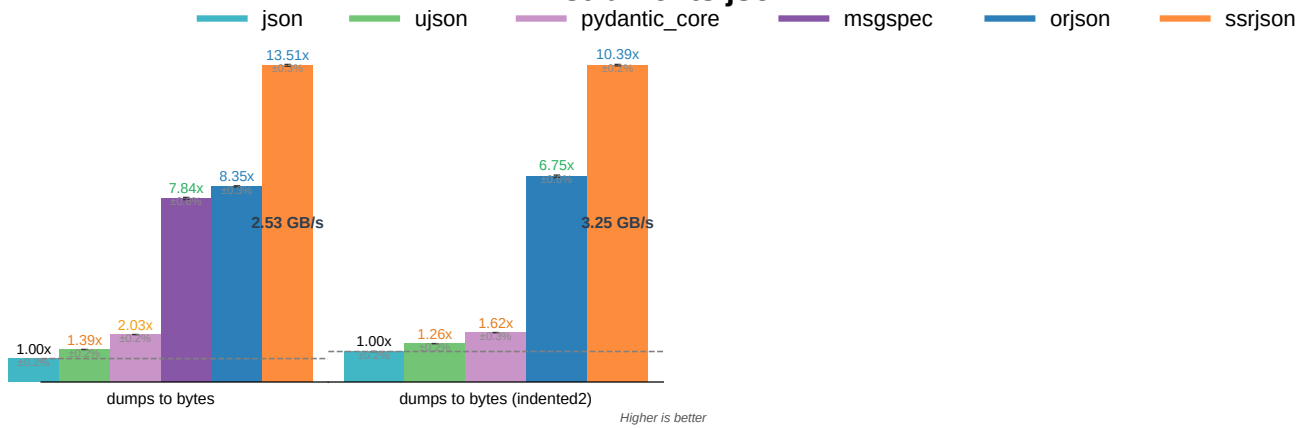
ctm.json



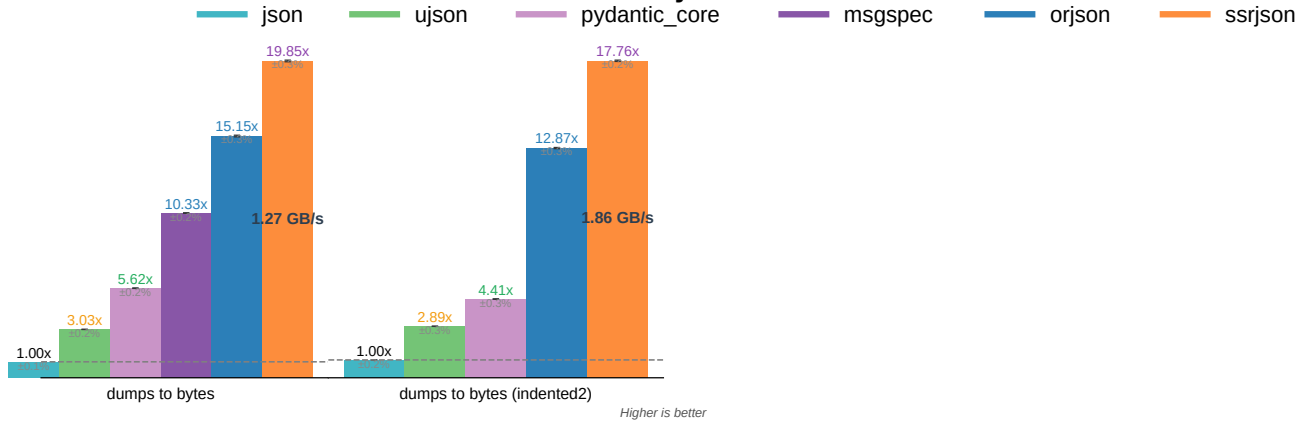
github.json



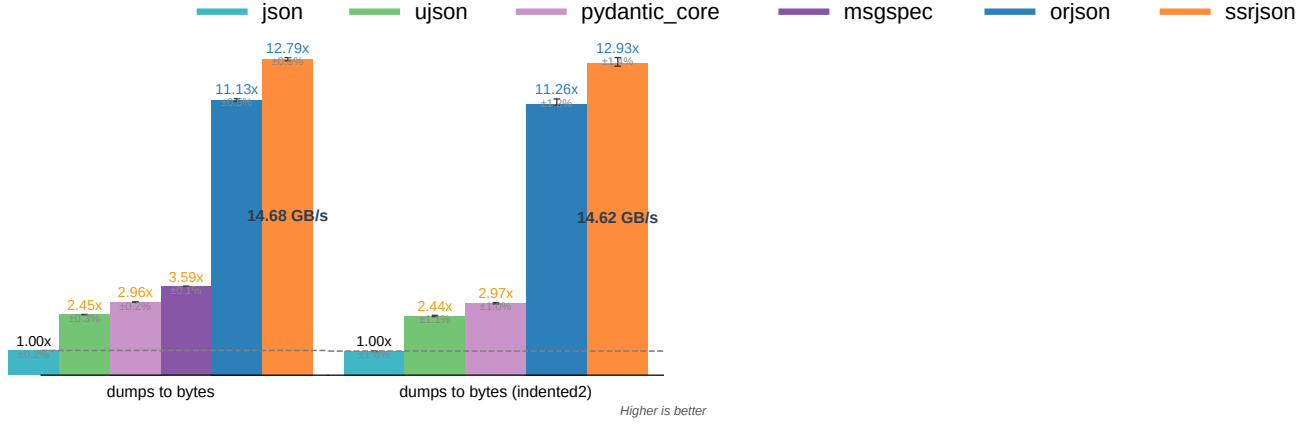
instruments.json



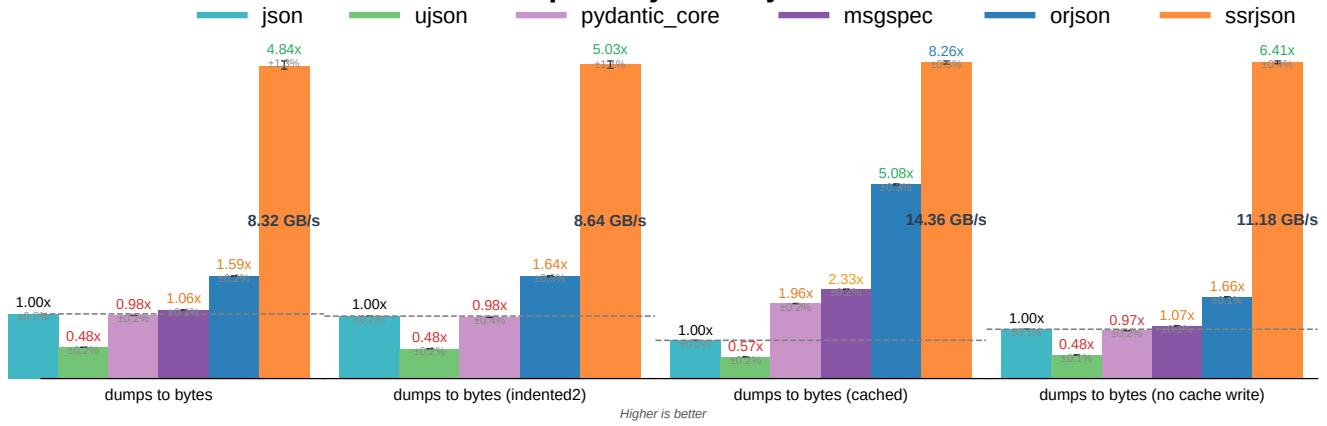
mesh.json



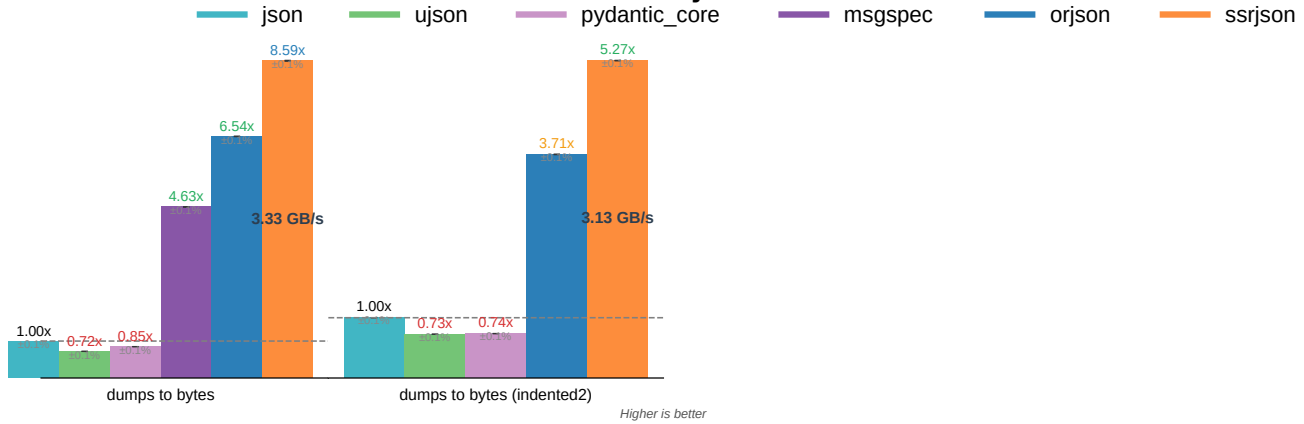
simple_object.json



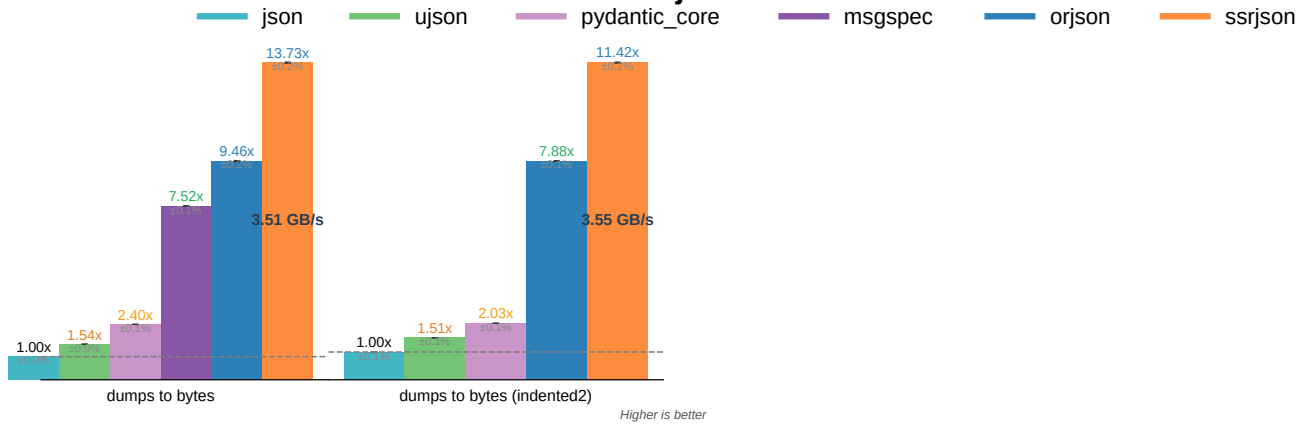
simple_object_zh.json



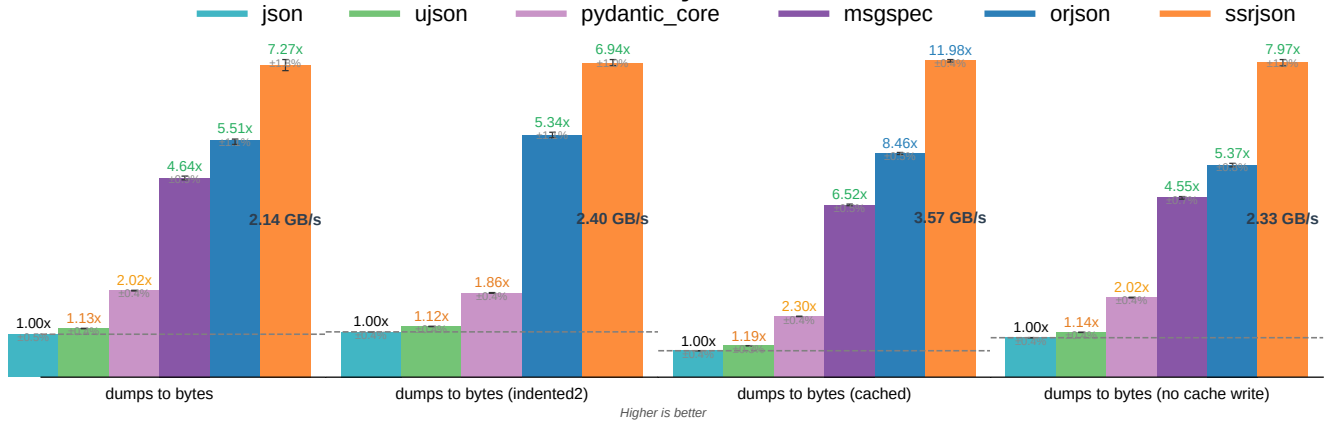
truenull.json



tweet.json

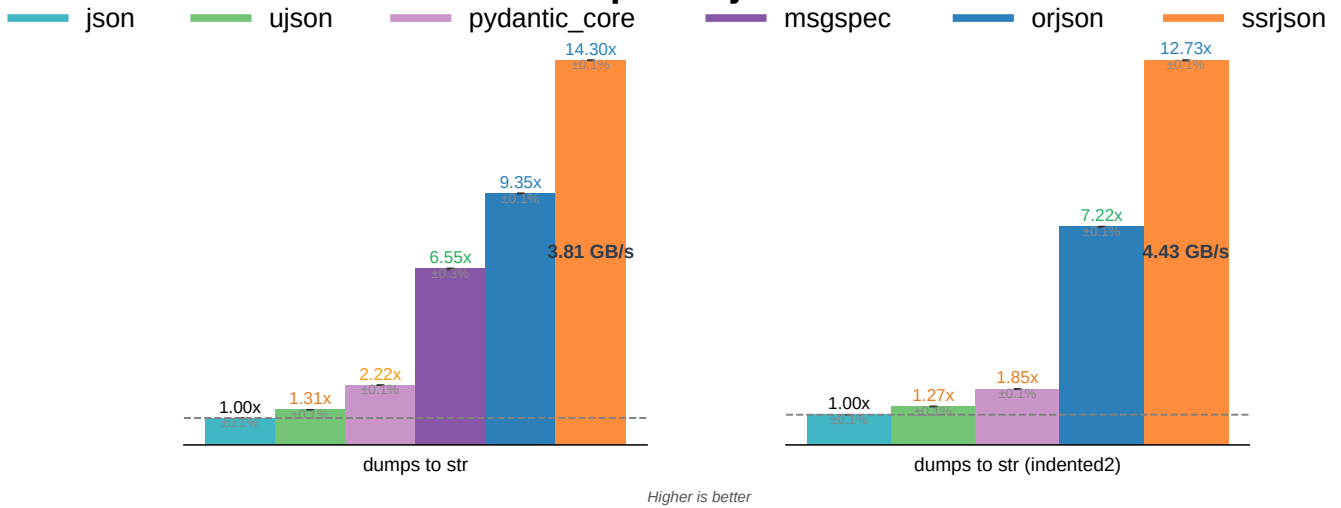


twitter.json

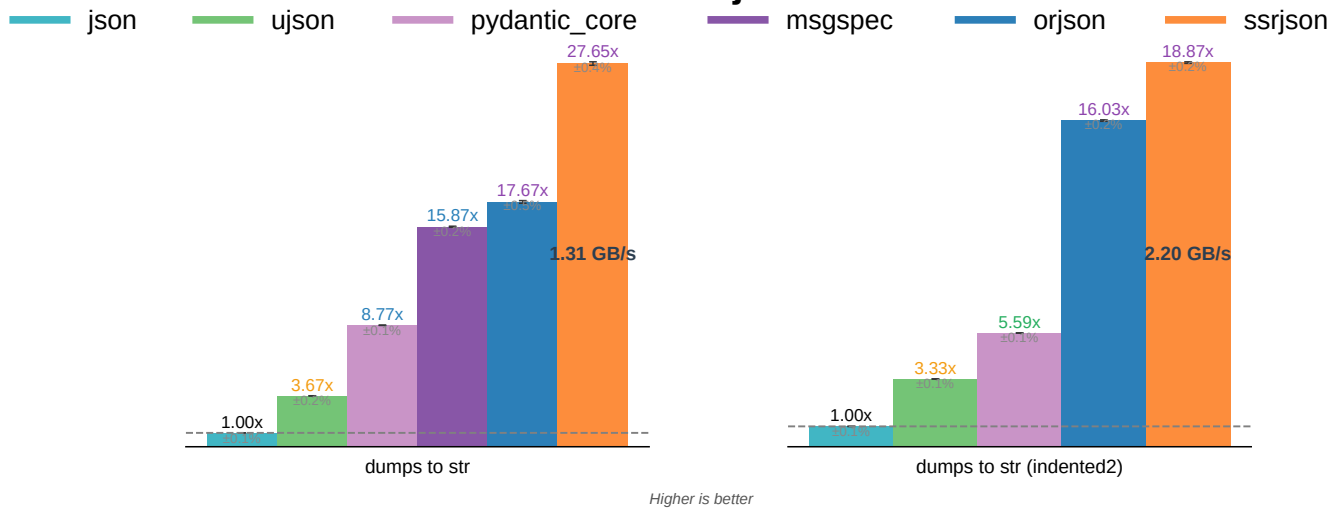


Dumps to str (hot)

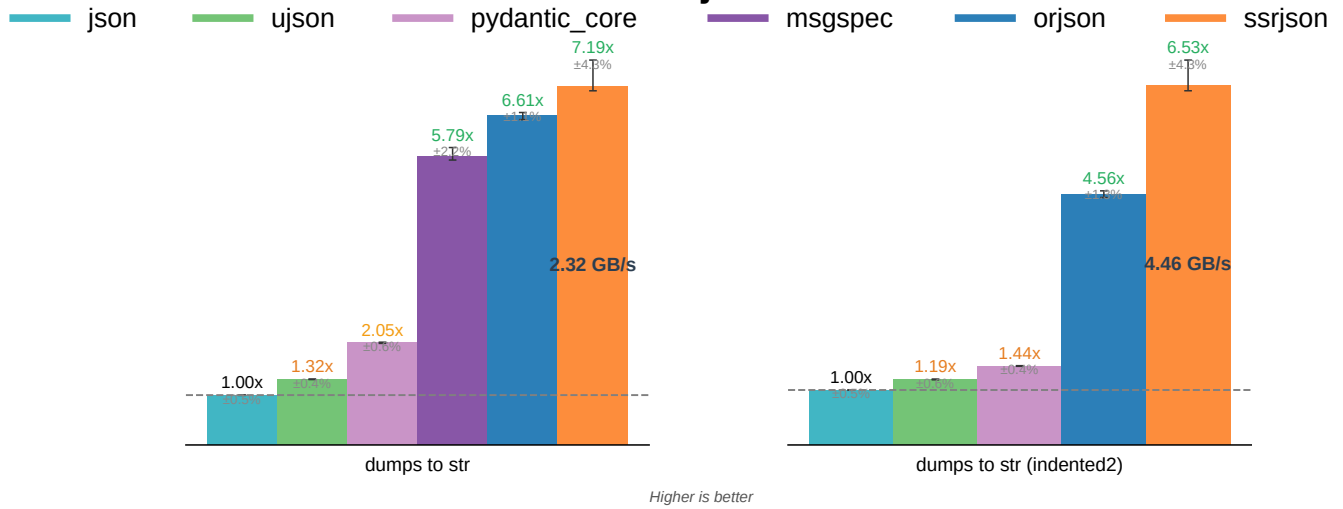
apache.json



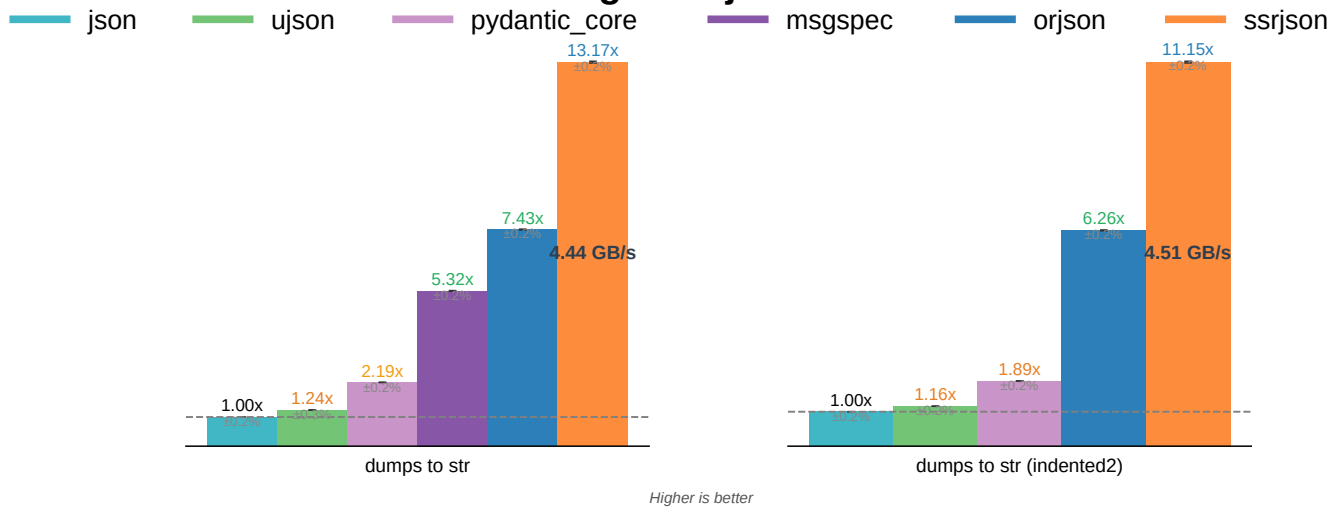
canada.json



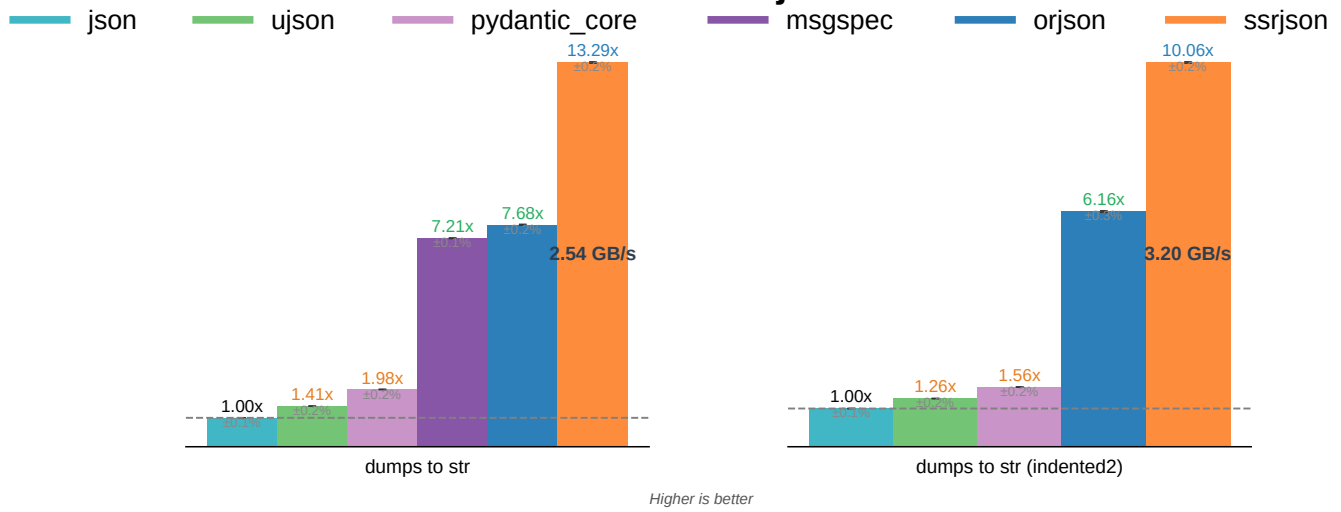
ctm.json



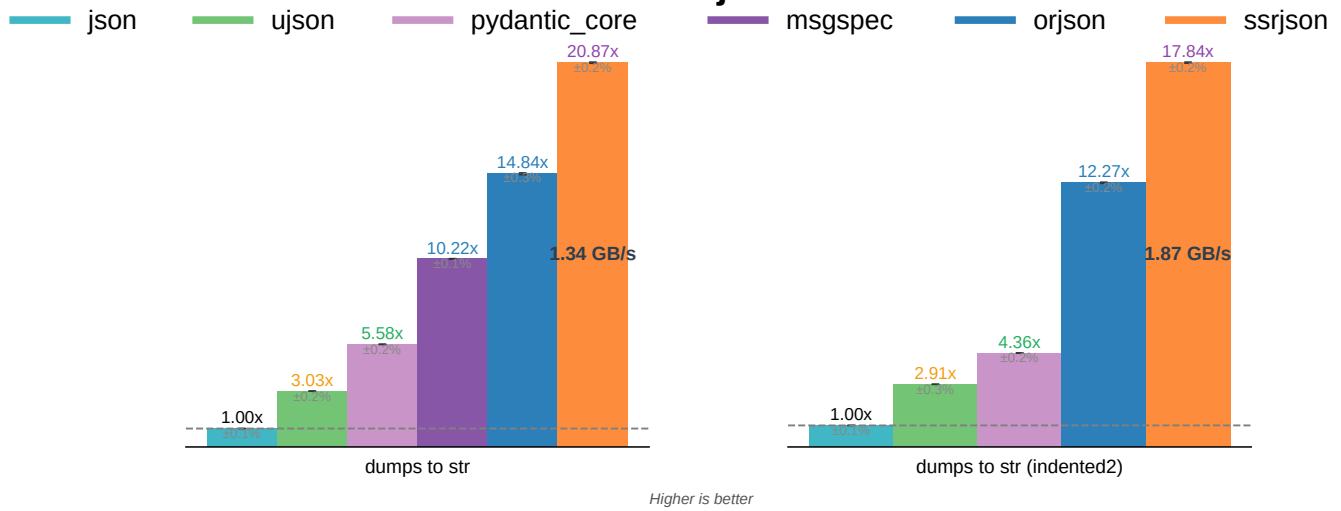
github.json



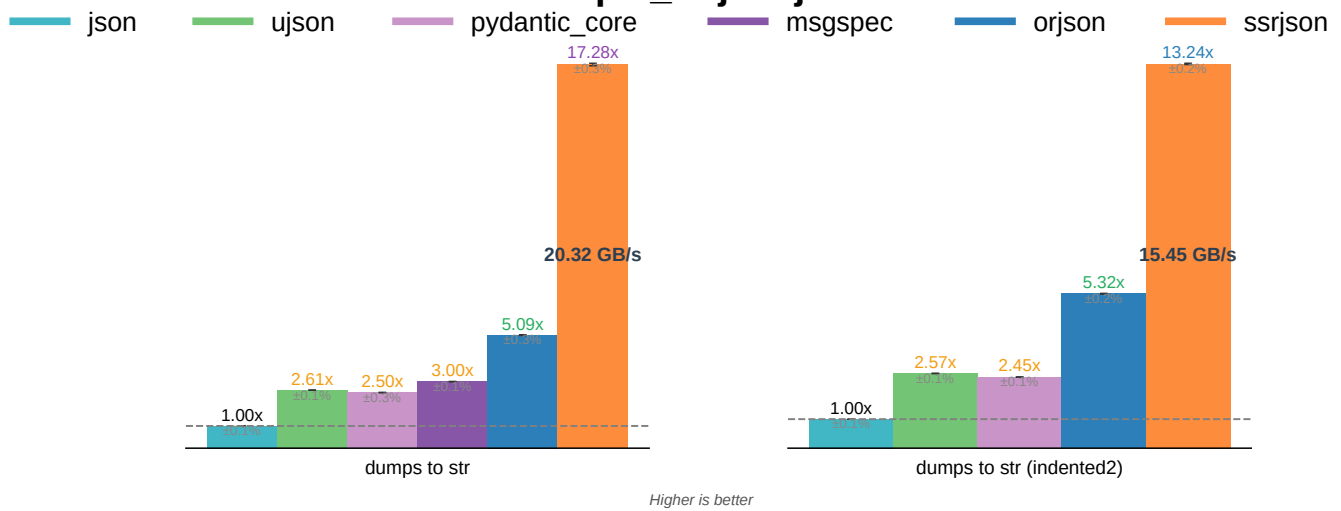
instruments.json



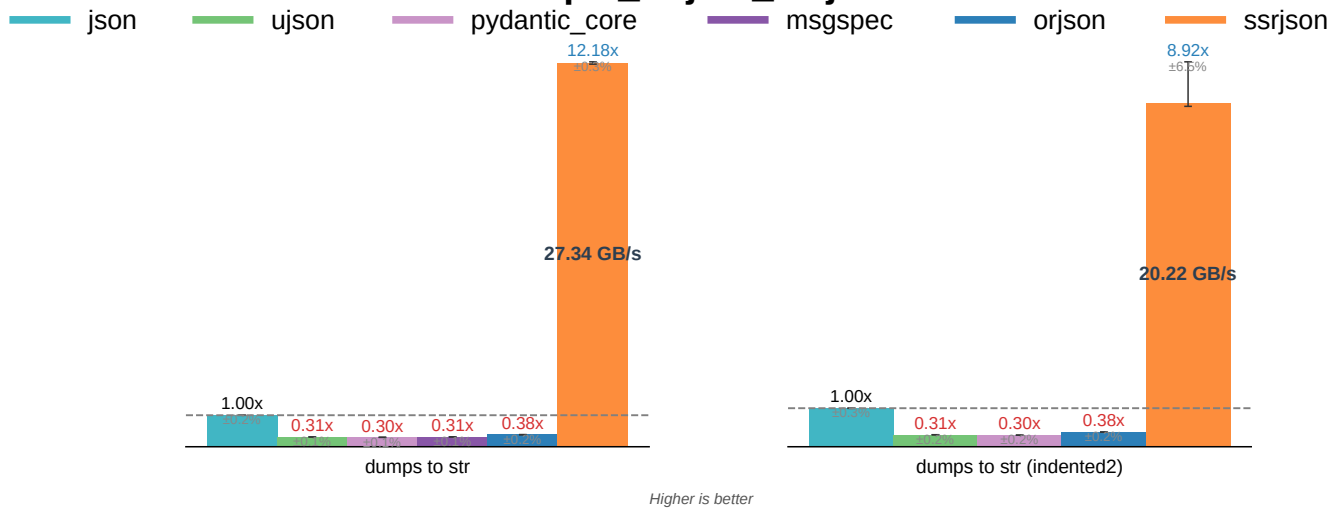
mesh.json



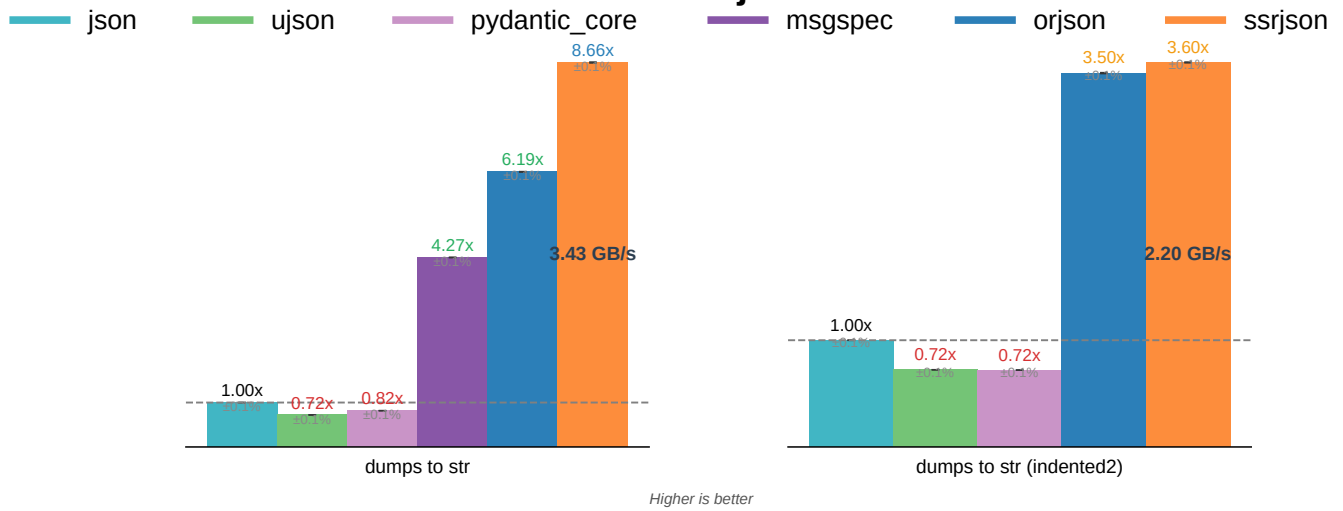
simple_object.json



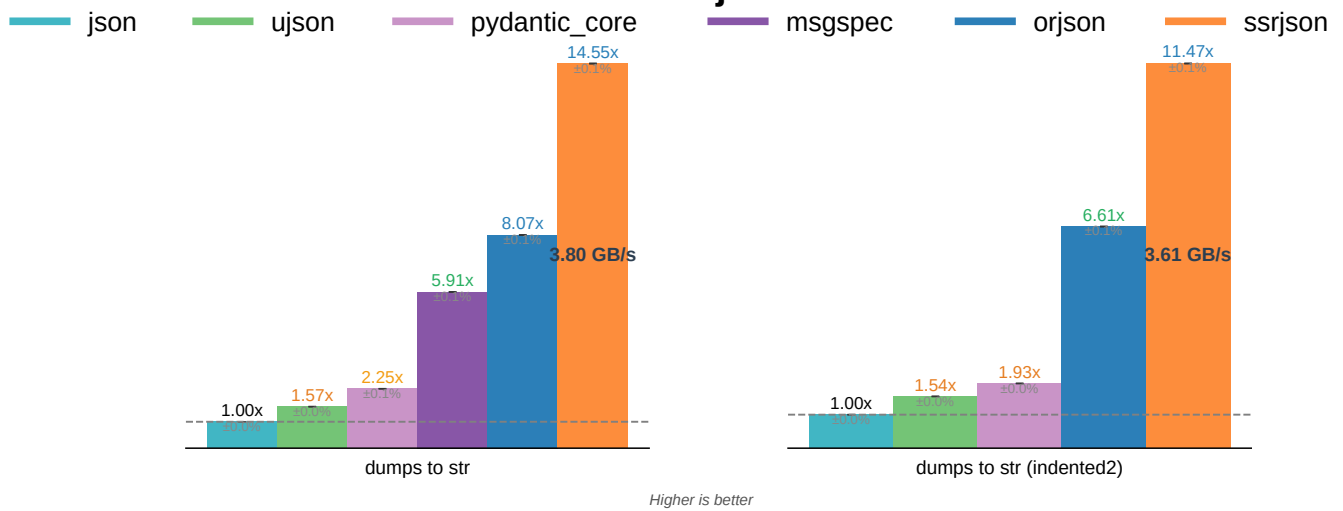
simple_object_zh.json



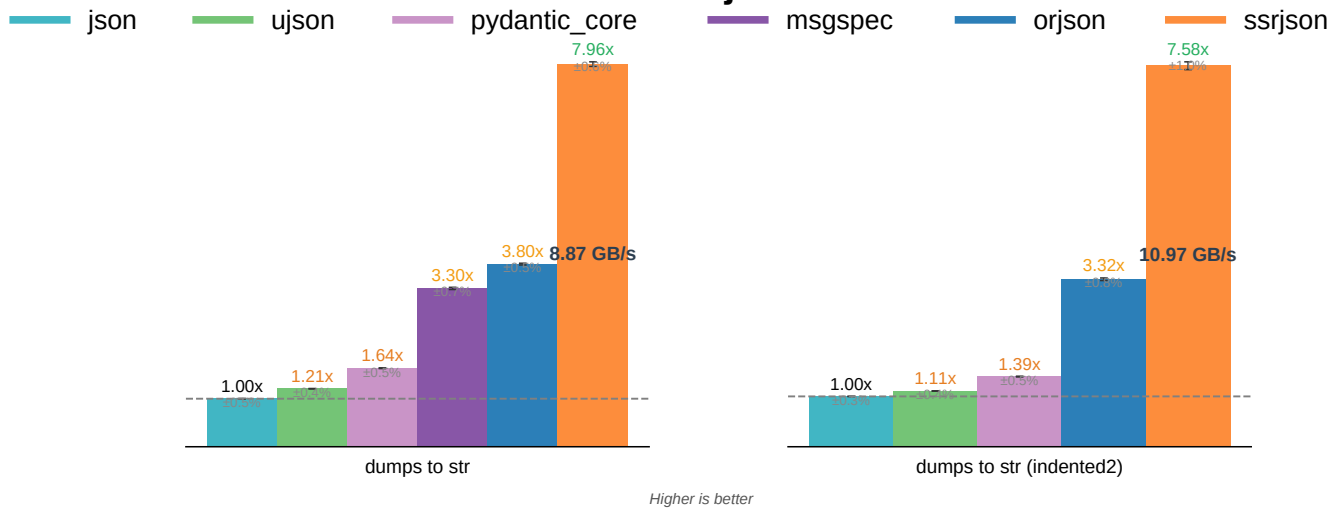
truenull.json



tweet.json

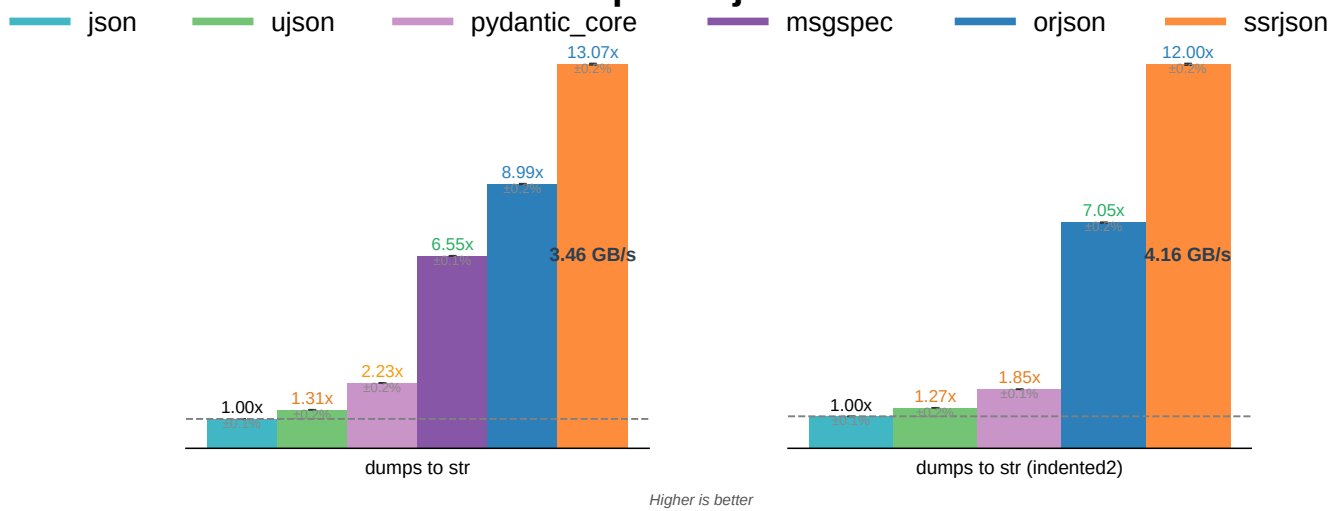


twitter.json

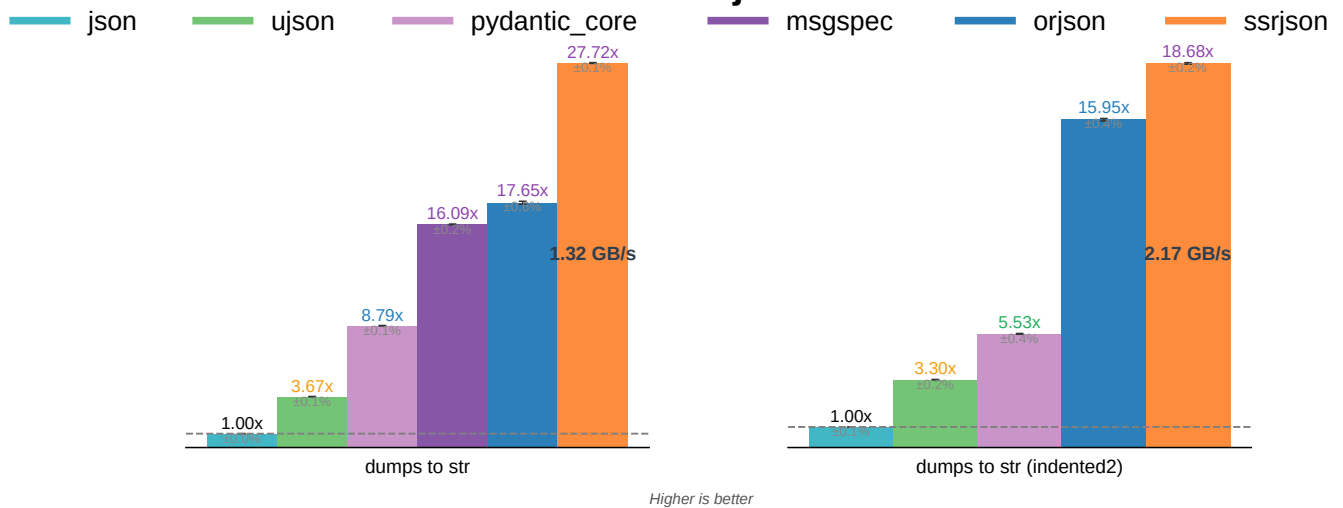


Dumps to str (cold)

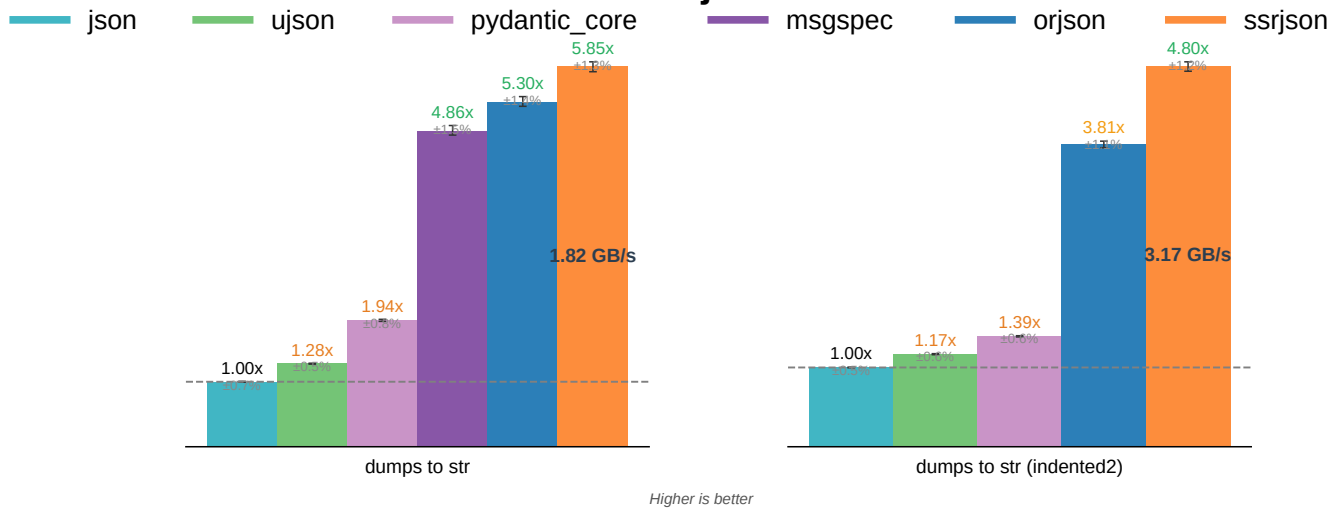
apache.json



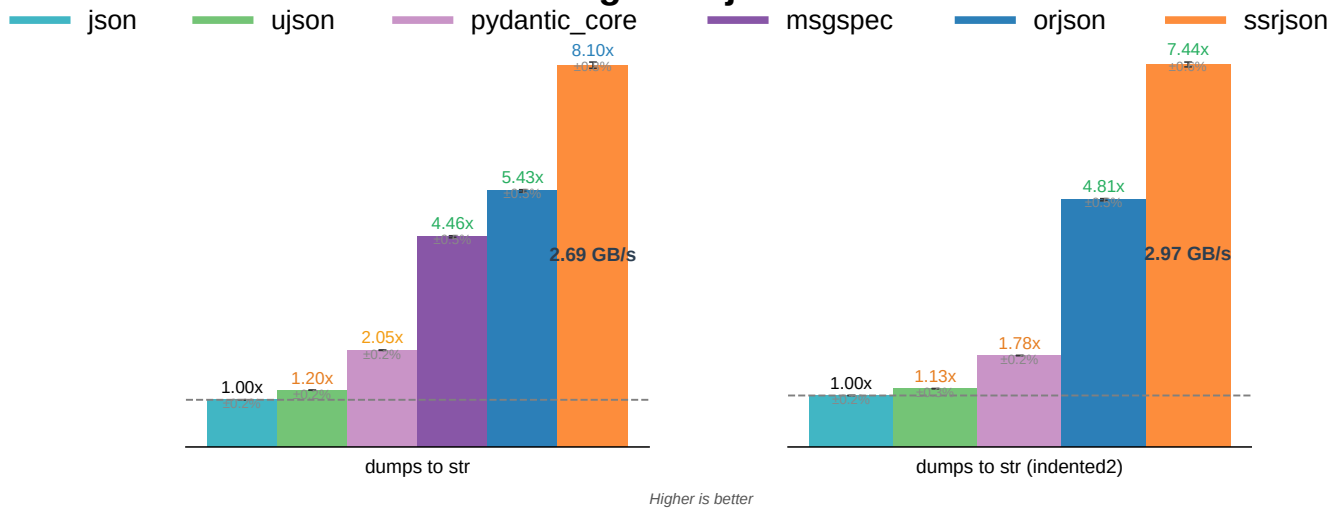
canada.json



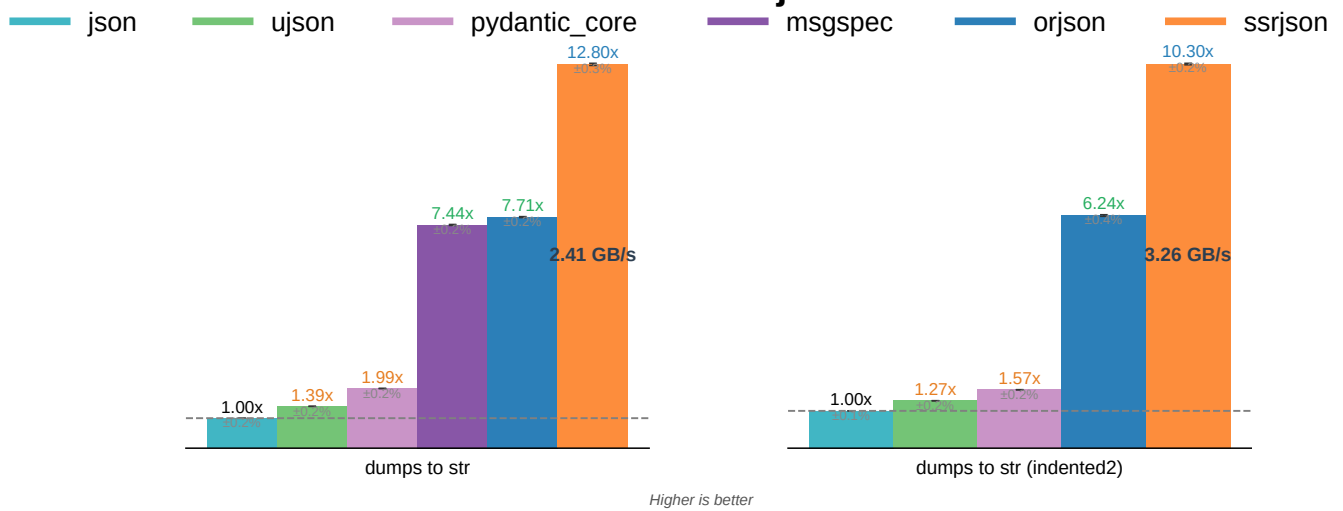
ctm.json



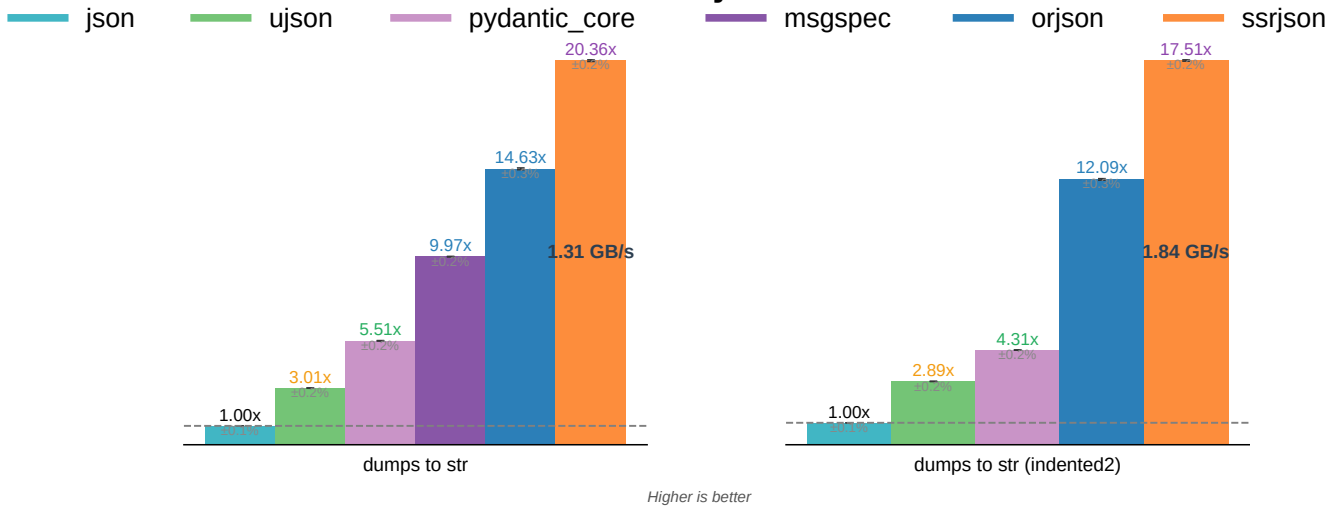
github.json



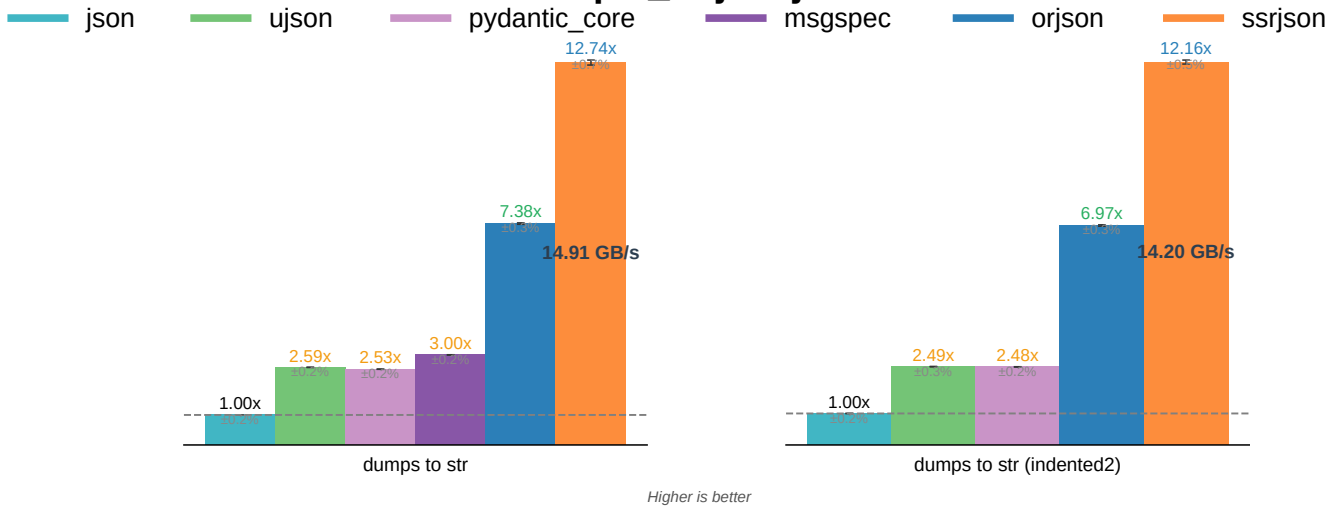
instruments.json



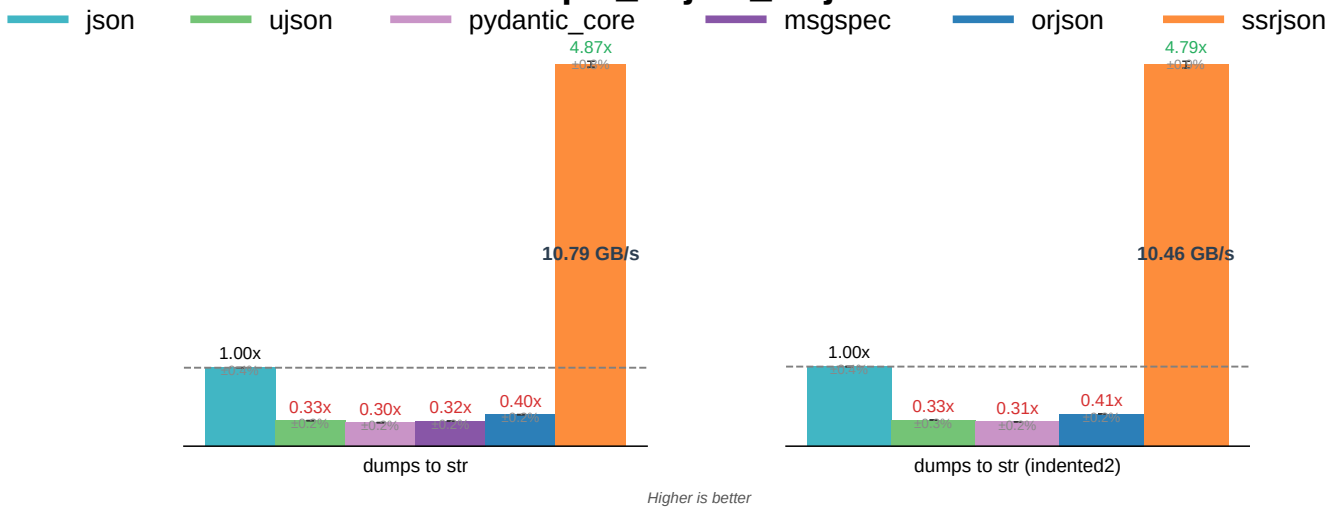
mesh.json



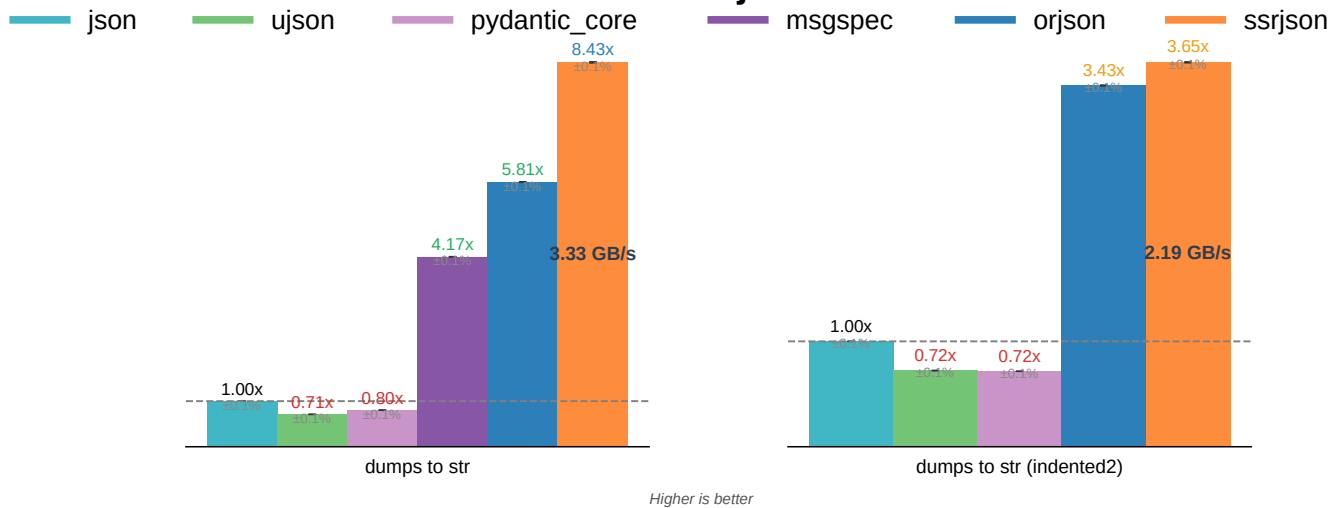
simple_object.json



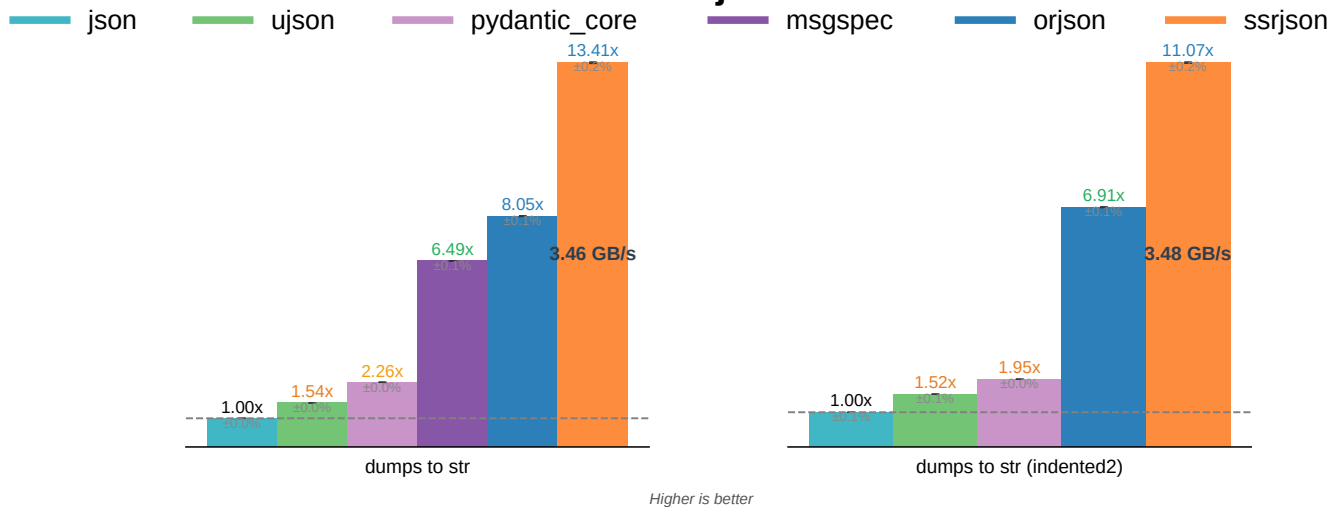
simple_object_zh.json



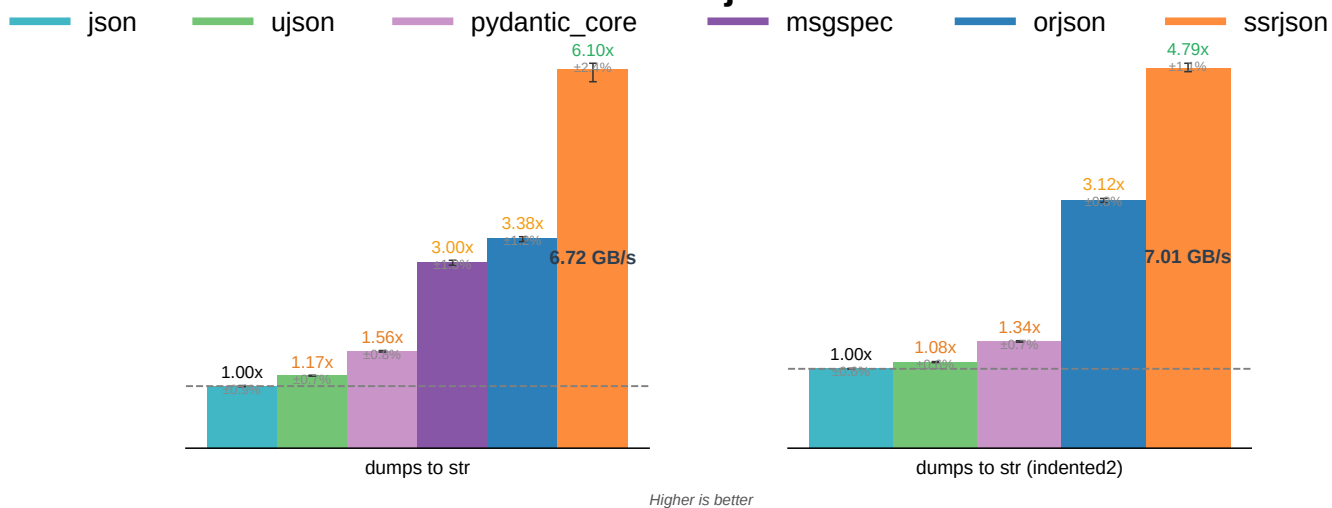
truenull.json



tweet.json



twitter.json

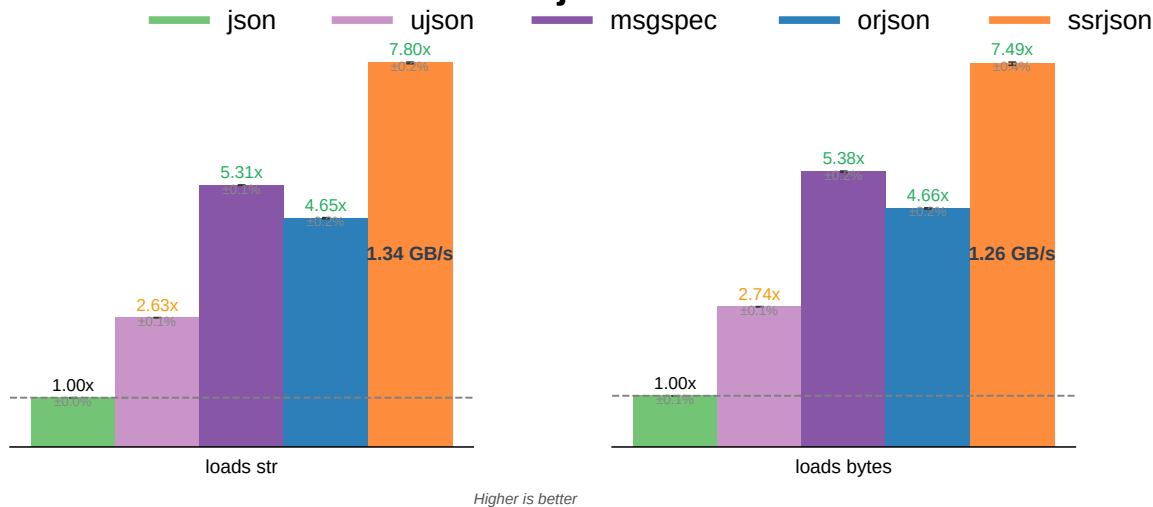


Loads (hot)

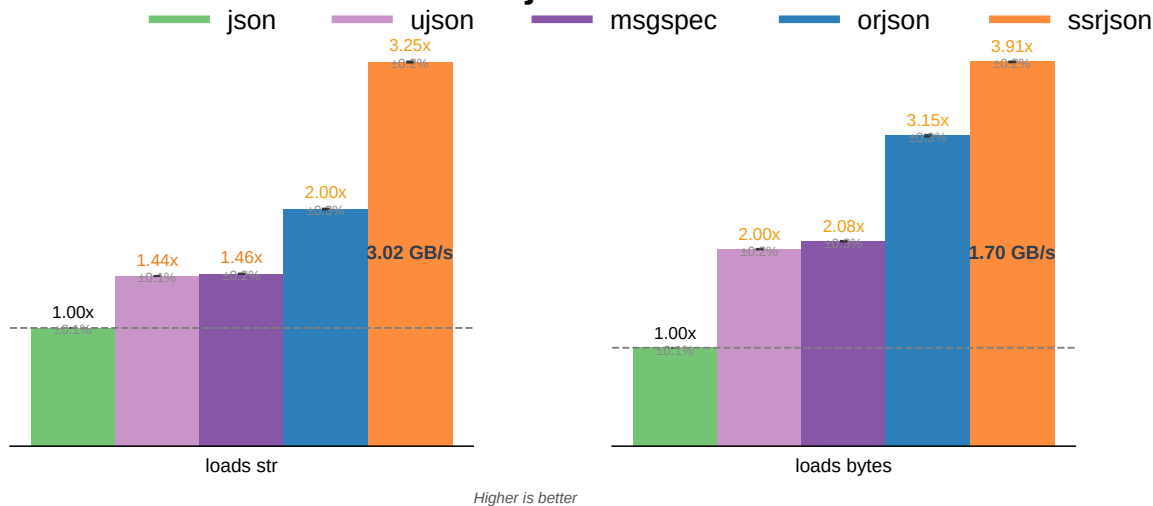
apache.json



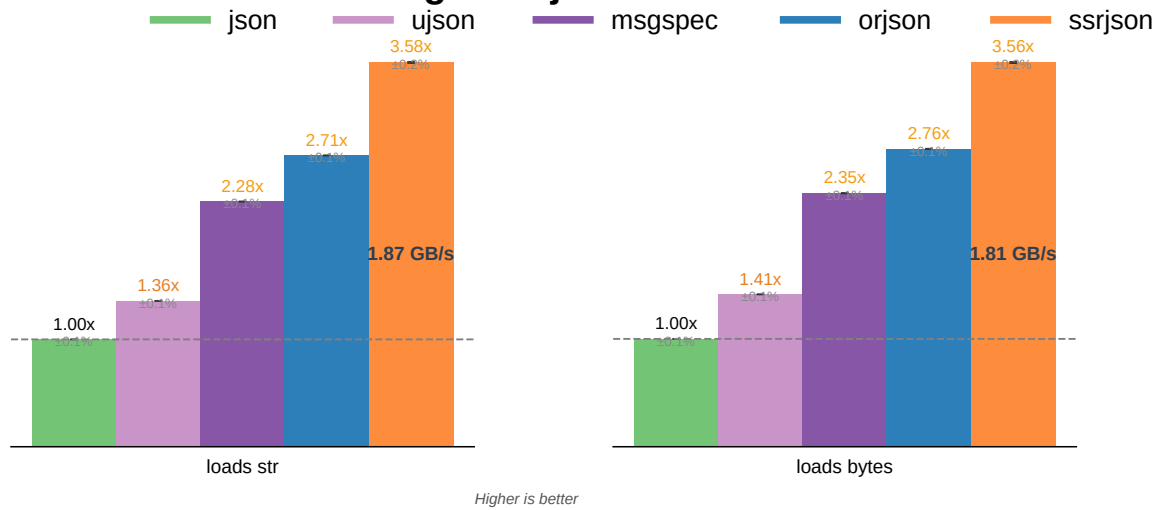
canada.json



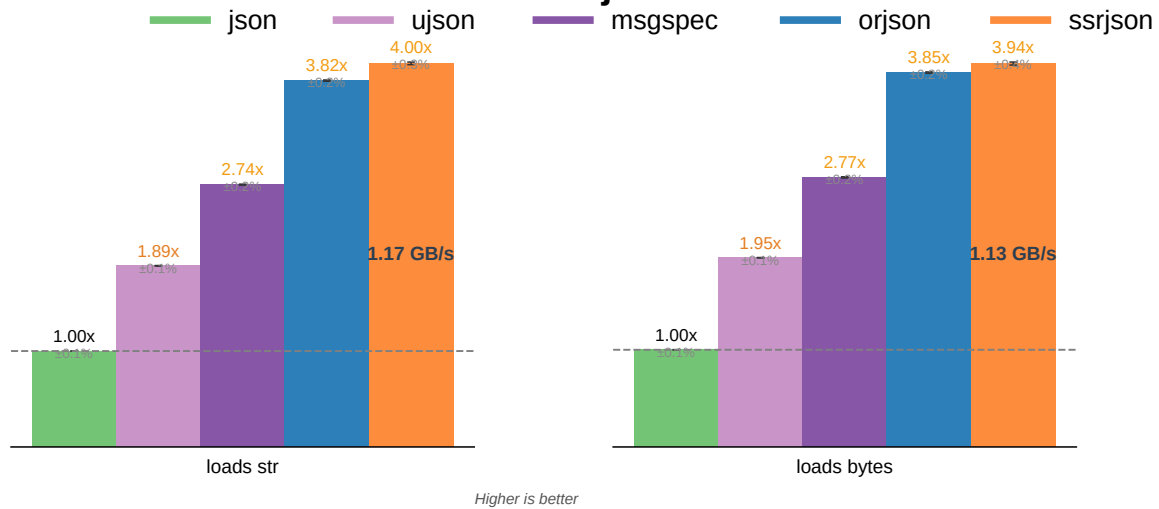
ctm.json



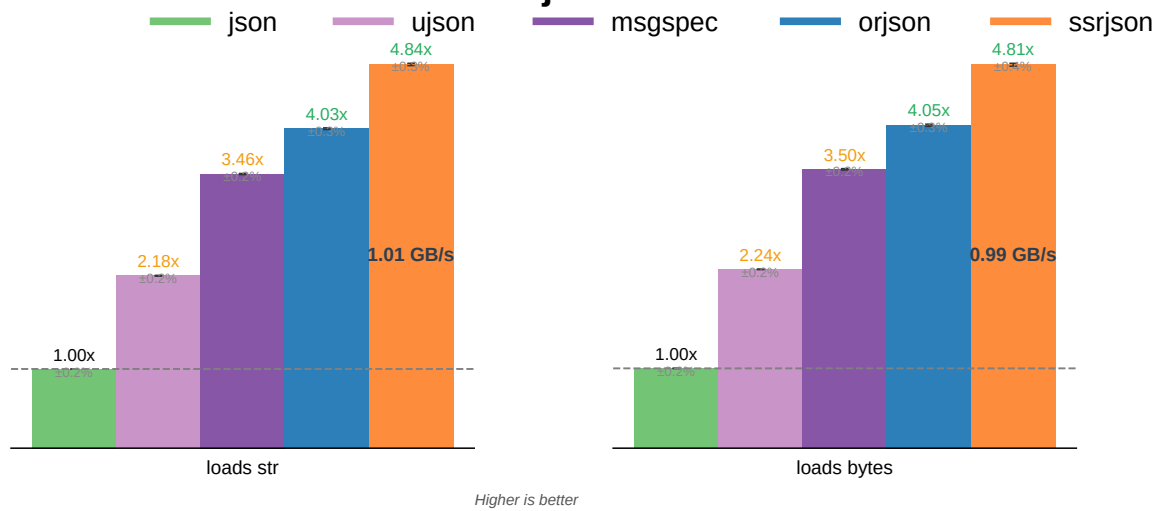
github.json

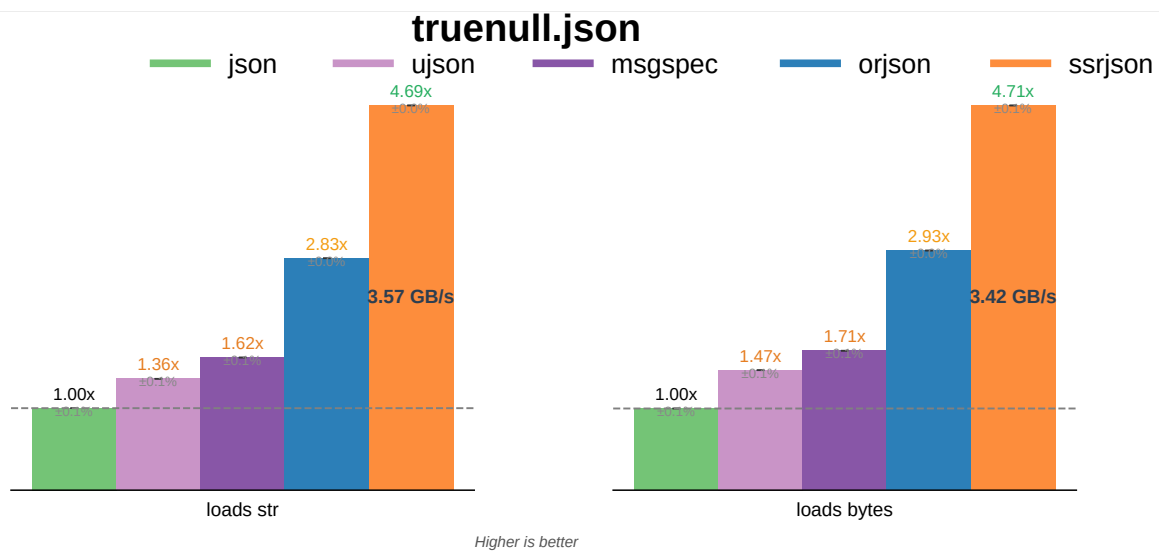
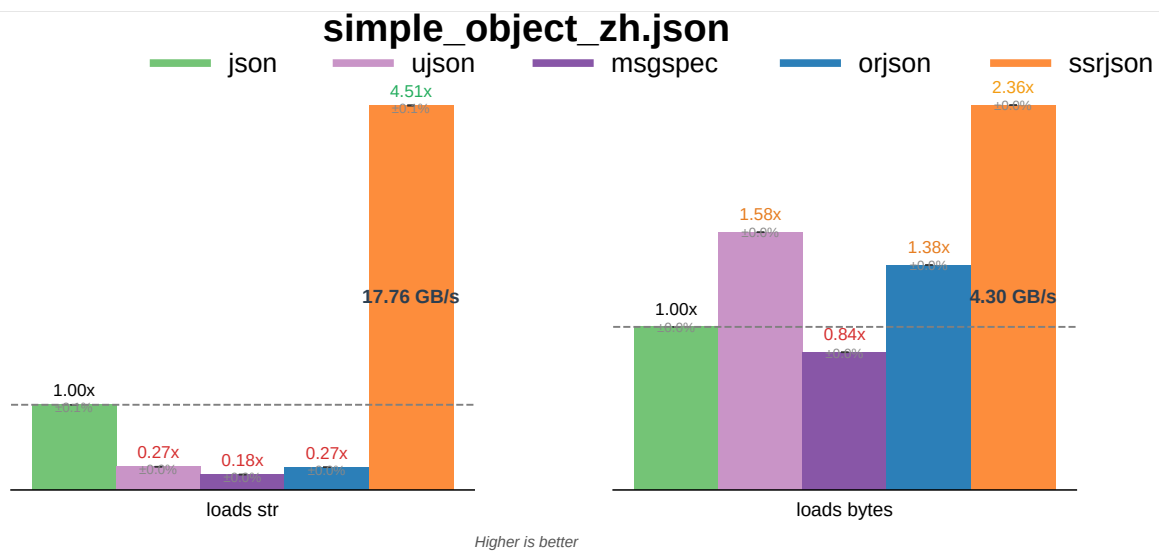
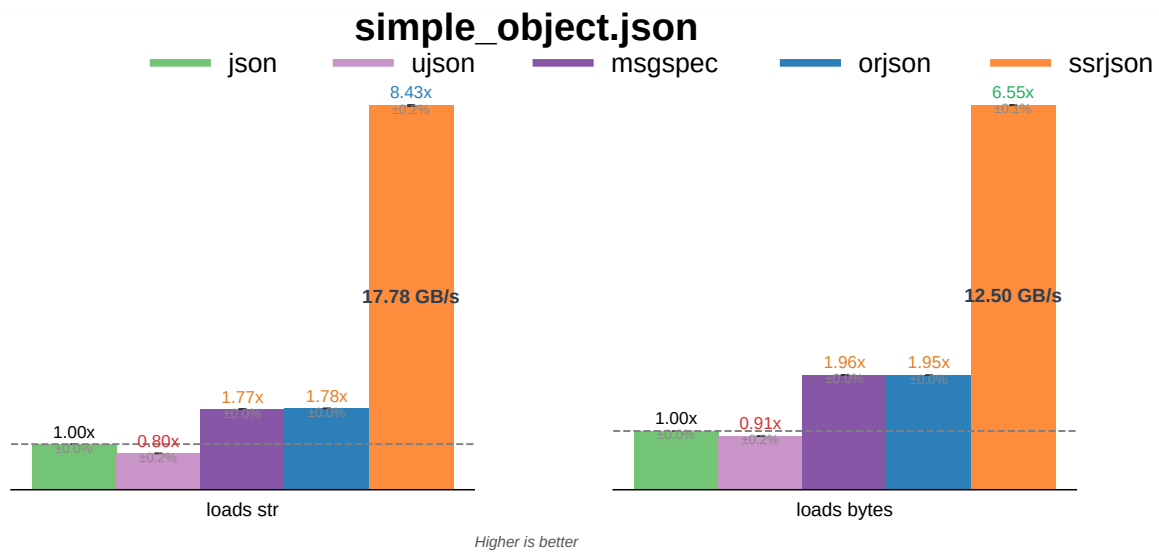


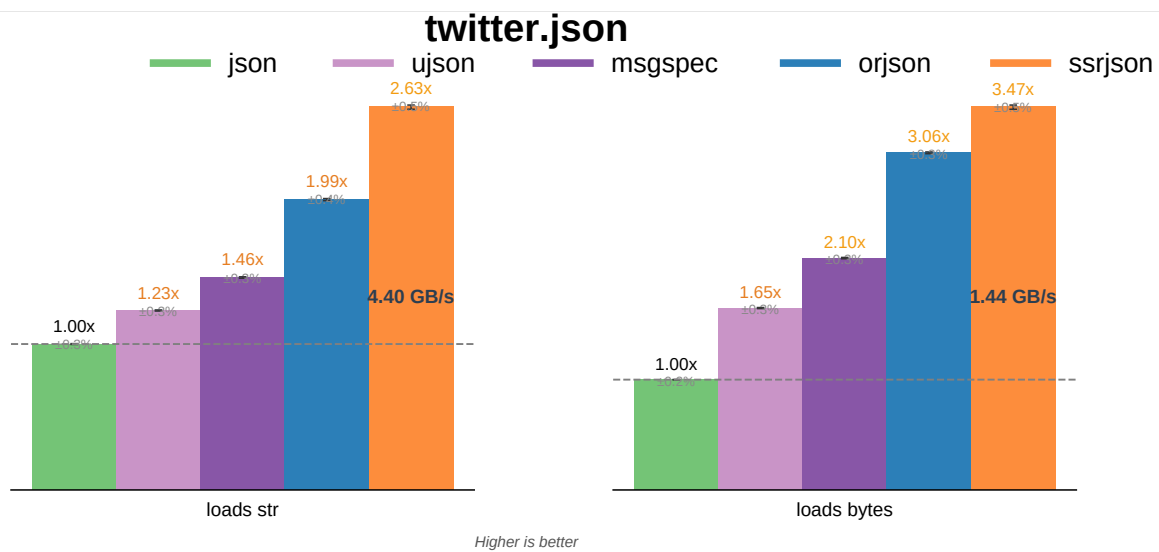
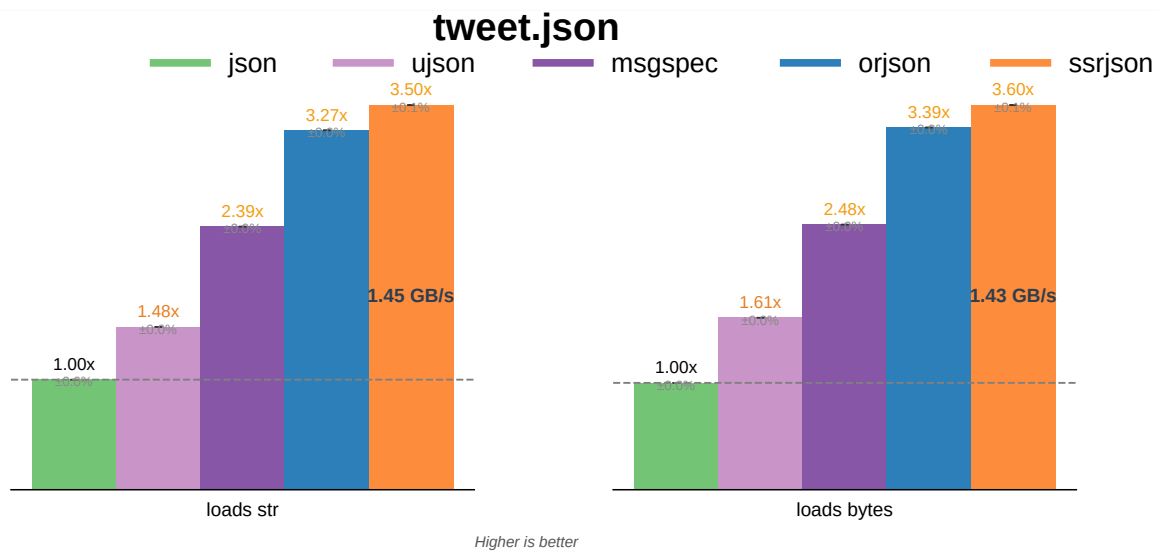
instruments.json



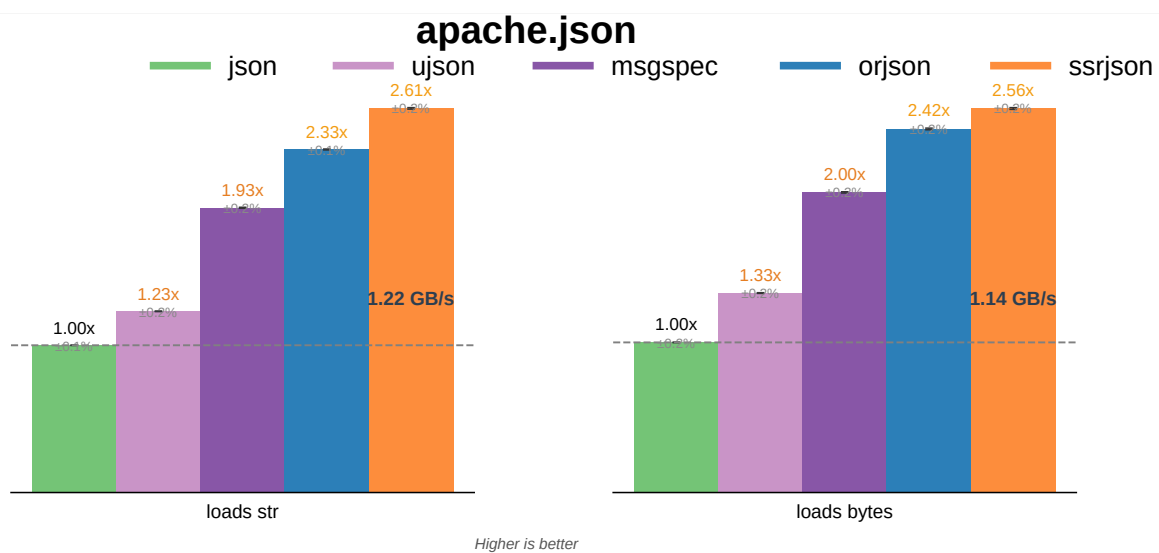
mesh.json



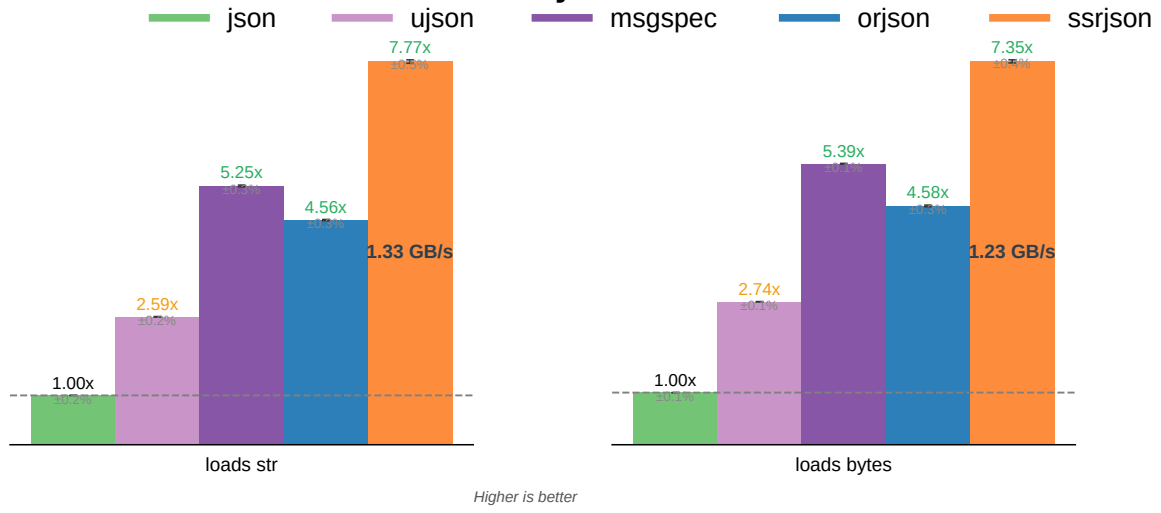




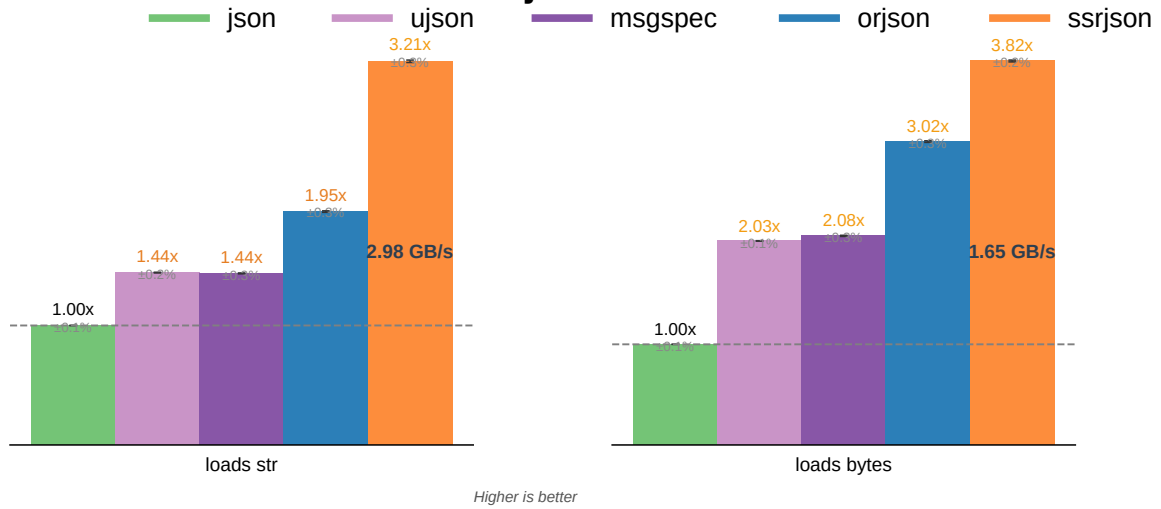
Loads (cold)



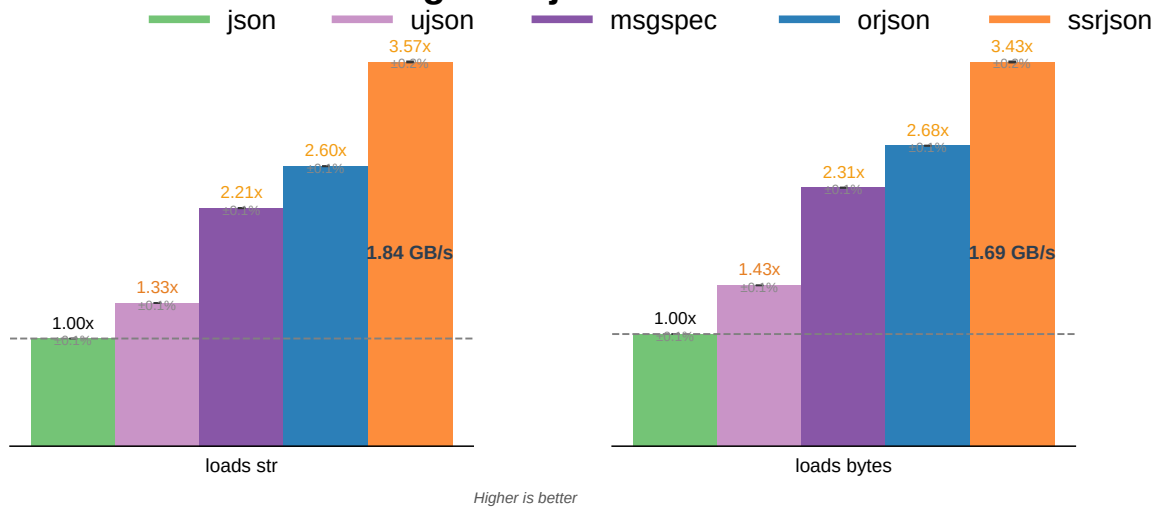
canada.json

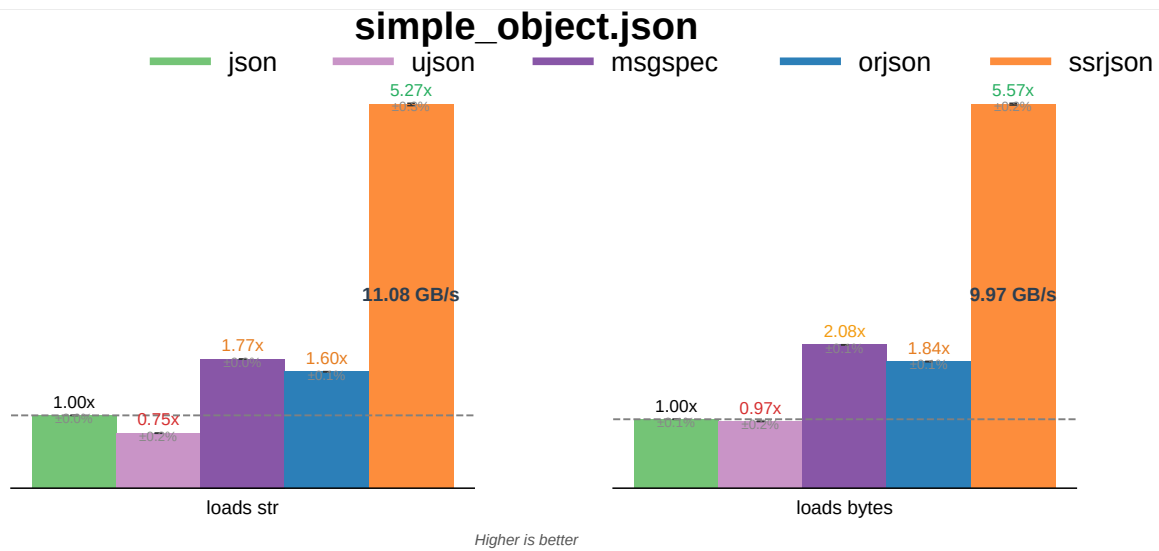
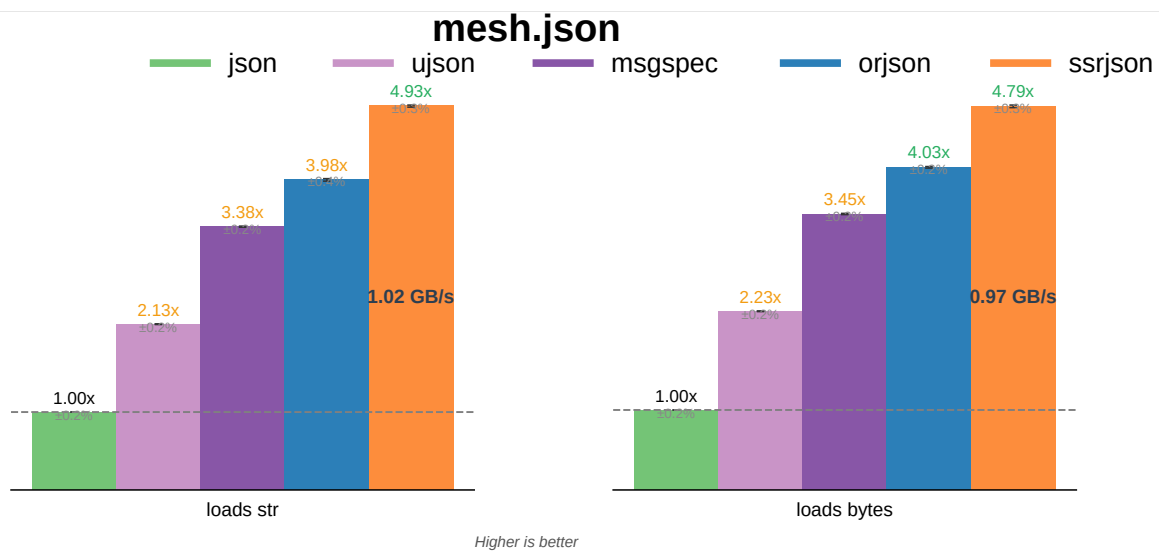
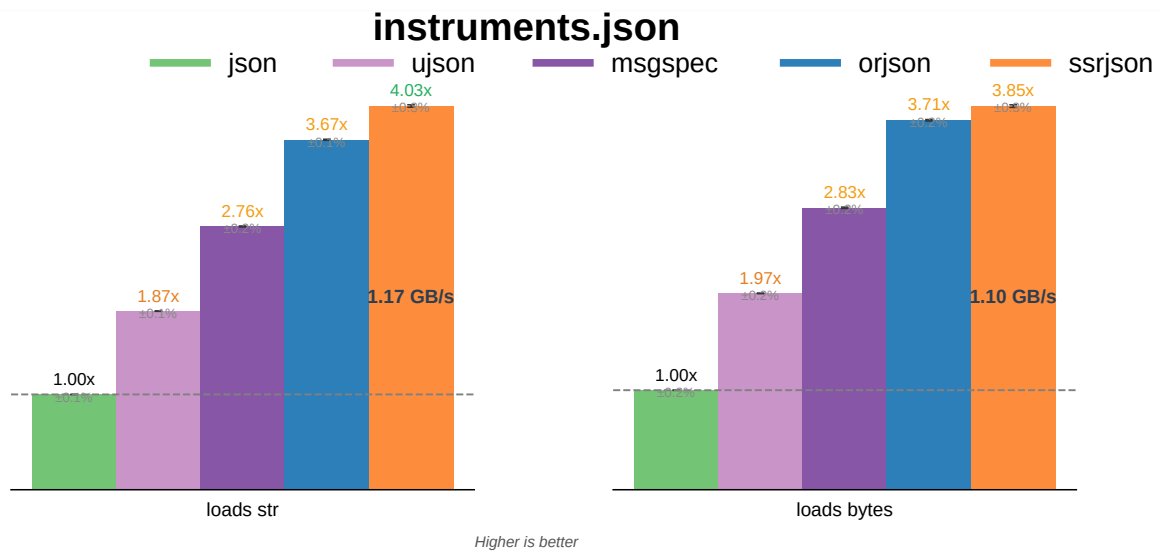


ctm.json

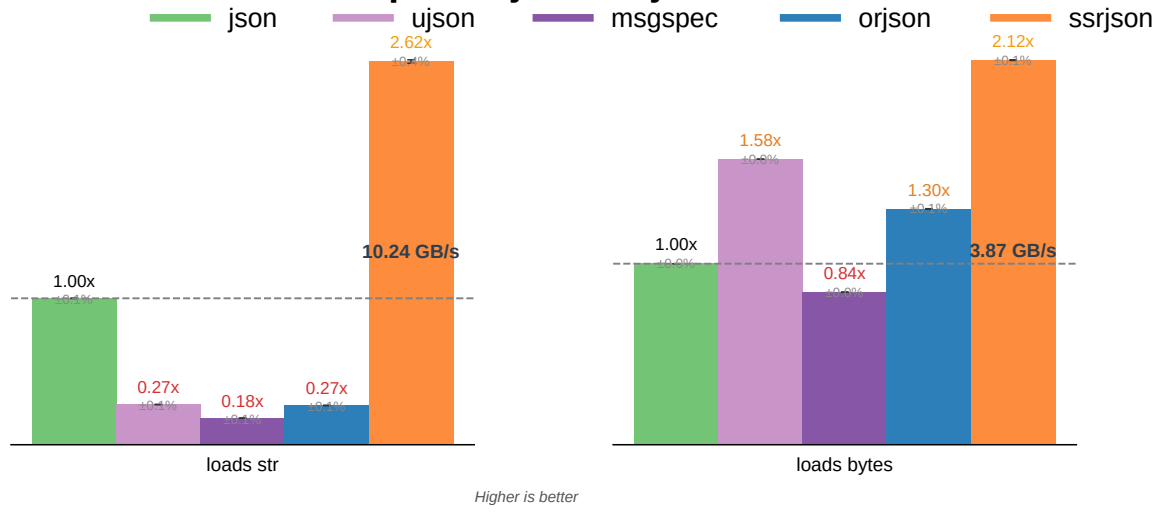


github.json





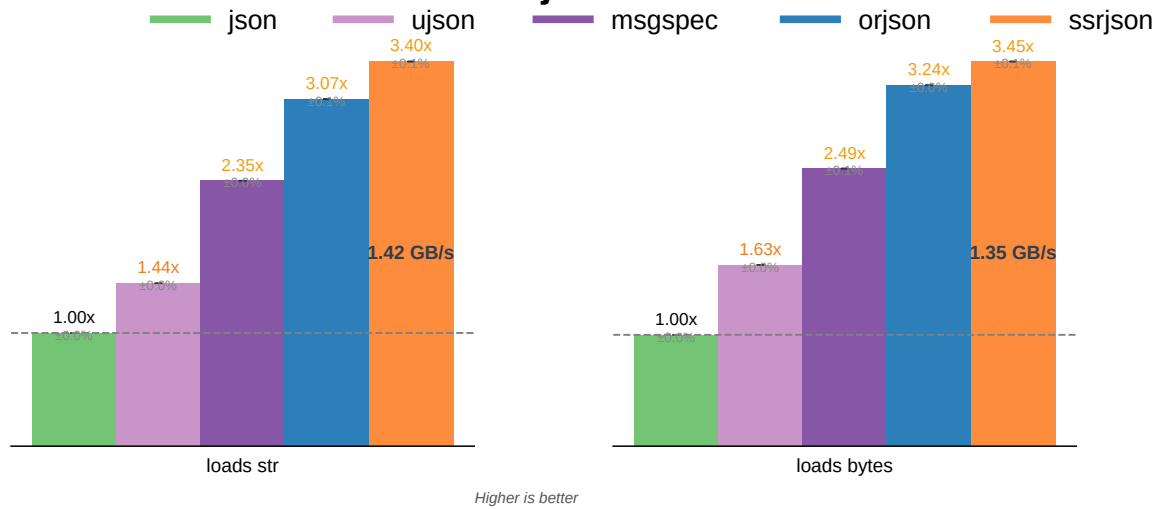
simple_object_zh.json



truenull.json



tweet.json



twitter.json

